

Credit Card Fraud Detection 분석 보고서

프로젝트: credit_card_fraud_detection 팀 프로젝트

분석 대상: Kaggle Credit Card Fraud Detection 데이터셋

팀원: 이해현 · 이진희 · 김소현 · 전유경 · 권용우

작성일: 2026-08-25 (최종 수정: 2026-09-02)

목차

0. 초록

1. 서론

- 1.1 연구 배경 및 필요성
- 1.2 과제 목적

2. 데이터셋 소개

- 2.1 데이터 출처 및 구성
- 2.2 데이터 특성 및 구조 분석
- 2.3 Feature 이름 특징
- 2.4 동일 값·중복 Feature
- 2.5 ID 변수
- 2.6 타깃 변수 분포 분석

3. 탐색적 데이터 분석(EDA)

- 3.1 기술통계 분석
- 3.2 상관관계 기반 주요 변수 분석
- 3.3 이상치 분석
- 3.4 변수별 이상치 시각화
- 3.5 중복 값 제거
- 3.6 특정 컬럼 로그 변환

4. 데이터 전처리

- 4.1 이상치 처리
- 4.2 분포 변환
- 4.3 파생 변수 생성
- 4.4 피처 정제 및 중요도 기반 가지치기
- 4.5 스케일링 및 차원 축소
- 4.6 데이터 분할

5. 분석방법(Analysis method)

- 5.1 전처리 필요성 및 과정 개요
- 5.2 사용 모델 설명
- 5.3 하이퍼파라미터 최적화 기법
- 5.4 실행환경
- 5.5 코드 재사용 및 함수화

6. 모델링

- 6.1 CatBoost (전유경)
- 6.2 LightGBM (이혜현)
- 6.3 Logistic Regression (김소현)
- 6.4 Logistic Regression (권용우)
- 6.5 XGBoost (이진희)

7. 결과

- 7.1 모델별 최종 성능 비교

7.2 최고 성능 모델 분석

7.3 하이퍼파라미터 최적화 효과 분석

7.4 피처 중요도 공통 인사이트

8. 종합 평가

9. 결론

10. 회고

10.1 잘된 점 · 아쉬운 점 · 향후 개선 방향

10.2 팀원별 느낀점 및 피드백

11. 참고문헌

12. 부록: 함수 코드

12.1 전처리 함수

12.2 공동 함수 — 모델링

12.3 공동 함수 — 하이퍼파라미터 최적화

12.4 공동 함수 — 평가

0. 초록

본 보고서는 Kaggle 에 공개된 Credit Card Fraud Detection 데이터셋(*creditcard.csv*, 284,807 건, PCA 로 익명화된 28 개 성분 V1~V28 + 거래 금액 Amount·경과 시간 Time·타겟 Class)을 이용해 신용카드 거래의 사기 여부를 예측하는 이진 분류 프로젝트를 정리한 것이다. 사기 거래가 전체의 0.173%(492 건)에 불과한 극단적 클래스 불균형 때문에 단순 정확도로는 성능을 가늠할 수 없고, 전처리와 평가지표 선택이 프로젝트 전체의 쟁점이 되었다. 팀원 5 명이 CatBoost(전유경), LightGBM(이혜현), XGBoost(이진희), Logistic Regression(김소현·권용우) 다섯 파이프라인을 각자 독립적으로 실험하고, 수동/격자 탐색과 Hyperopt(TPE)를 나란히 비교했다. 주요 결과는 (1) 완전 중복행 제거와 "분할 → 전처리" 순서 같은 방법론적 원칙이 모델 선택보다 성능의 신뢰도를 좌우했고, (2) ROC-AUC 하나만으로는 오탐 비용이 드러나지 않아 PR-AUC 와 F1 을 함께 확인해야 했으며, (3) PR-AUC 기준으로 트리 기반 부스팅(CatBoost·XGBoost·LightGBM)이 로지스틱 회귀보다 앞서는 경향이 다섯 실험 전반에서 공통으로 확인되었다는 것이다. 다만 팀원별 전처리·검증 조건이 달라 세 부스팅 모델 간 우열은 단정하지 않았다.

1. 서론

1.1 연구 배경 및 필요성

신용카드 이상거래 탐지는 은행·카드사가 정상 거래 흐름을 방해하지 않으면서 소수의 사기 거래를 잡아내야 하는 실무 과제다. 사기를 놓치면(미탐지, FN) 금전 피해와 고객 신뢰 손상으로 이어지고, 정상 거래를 사기로 잘못 차단하면(오탐, FP) 고객 불편과 상담 비용이 발생하므로, 두 오류의 비용을 함께 고려한 탐지 품질이 중요하다. 이번 프로젝트에서 다룬 *creditcard.csv* 데이터셋은 2013년 9월 유럽 카드 소지자의 이틀간 거래 284,807건으로, 개인정보 보호를 위해 원본 피처가 PCA로 변환된 28개 익명 성분(V1~V28)에 거래 금액(Amount)과 경과 시간(Time), 타깃(Class)만 원본 형태로 남아 있다. 사기 거래는 492건(0.173%)에 불과해 “모든 거래를 정상으로 예측”하기만 해도 정확도(accuracy)가 99.8%를 넘는다. 즉 이 데이터에서는 정확도가 성능 지표로서 의미를 잃고, 어떤 지표로 평가하고 어떤 전처리·모델을 선택하느냐가 결과를 가른다.

1.2 과제 목적

본 프로젝트의 목적은 팀원 5명이 서로 다른 전처리 전략과 모델을 각자 담당해 실험한 뒤, 이를 하나의 파이프라인 관점에서 비교·정리하는 것이다. 전유경은 CatBoost, 이해현은 LightGBM, 이진희는 XGBoost를 맡았고, 김소현과 권용우는 같은 Logistic Regression을 서로 다른 방식으로 다뤘다 — 김소현은 *class_weight*와 SMOTE 비교, L1 정규화를 통한 변수 자동 선택, 평가지표를 PR-AUC로 바꾼 규제 강도(C) 재탐색에, 권용우는 8가지 전처리 조합 전수 비교와 GridSearchCV·Hyperopt·SMOTE의 동일 조건 비교에 무게를 뒀다. 같은 원본 데이터를 쓰되 전처리·모델·튜닝 전략은 팀원마다 달랐고, 그 차이 자체가 “어떤 조합이 이 데이터에 실제로 효과적인가”를 판단하는 근거가 된다. 절차는 (1) 데이터 이해 및 EDA, (2) 데이터 전처리, (3) 모델링 및 비교, (4) 하이퍼파라미터 최적화, (5) 성능평가 및 해석, (6) 결론 및 회고의 순서로 진행되었으며, 본 보고서 역시 이 흐름을 따른다.

2. 데이터셋 소개

2.1 데이터 출처 및 구성

항목	내용
데이터 출처	Kaggle <i>Credit Card Fraud Detection</i> (Machine Learning Group – ULB), <i>creditcard.csv</i>
전체 건수	284,807 건
컬럼 수	31 개 (<i>Time</i> , <i>V1~V28</i> , <i>Amount</i> , <i>Class</i>)
피처 수(타깃 제외)	30 개 — 전부 수치형(<i>float64</i>)
결측치(NaN)	없음 (전 팀원이 <i>isnull().sum()</i> 으로 공통 확인)
클래스 분포	0(정상) 284,315 건(99.827%) / 1(사기) 492 건(0.173%) → 약 578 : 1 불균형
평가지표	ROC-AUC(보조) + PR-AUC / F1-Score(주)

산탄데르 데이터셋과 달리 별도의 *ID* 컬럼도, 제출용 테스트 파일도 없다. 하나의 *creditcard.csv*를 팀 전원이 동일하게 내려받아 *train_test_split*으로 나눠 썼다(자세한 분할 방식은 4.6).

2.2 데이터 특성 및 구조 분석

- **V1~V28**: 개인정보 보호를 위해 이미 PCA(주성분분석)로 변환된 익명 피처다. 평균이 거의 0 이고 서로 상관관계가 거의 없어(직교) PCA 변환의 전형적 특징을 보인다. 이 때문에 “어떤 거래 속성인지”를 알 수 없어 도메인 기반 피처 엔지니어링은 불가능하며, feature importance 로 “V14 가 중요하다” 정도의 통계적 해석만 가능하다.
- **Amount**: 결제 금액(원본 스케일). 평균(약 88)이 중앙값(약 22)의 4 배에 이를 만큼 오른쪽으로 크게 치우쳐(skewed) 있고, 소수의 고액 거래가 분포를 끌어당긴다.
- **Time**: 첫 거래 이후 경과 초(0~172,792 초, 약 이틀)로, 실제 시각(시/분)이 아니라 사기 여부와 직접적인 인과관계를 갖기 어려운 값이다.
- **Class**: 타깃 변수(0=정상, 1=사기).

기술통계량은 원본 *creditcard.csv* 기준이며 팀 전원이 *describe().info()*로 공통 확인했다. 전체 30 개 피처는 *float64*, *Class* 만 *int64* 다. 아래 표에는 원본 스케일 변수인 *Time*-*Amount* 만 싣고, PCA 성분과 타깃은 표 아래에 따로 정리한다.

변수	mean	std	min	25%	50%	75%	max
Time	94,814	47,488	0	54,202	84,692	139,320	172,792
Amount	88.35	250.12	0.00	5.60	22.00	77.16	25,691.16

V1~V28 은 PCA 로 변환된 28 개 성분으로 개별 의미를 알 수 없어 표에는 싣지 않고 공통 특성만 정리한다. 28 개 모두 평균과 중앙값이 0 에 수렴하며, 표준편차만 *V1*(1.96)에서 *V28*(0.33)로 단조 감소한다(주성분 순서 = 분산 설명력 순서). 다만 일부 성분은 꼬리가 매우 길어 최솟값은 *V5* 의 -113.74, 최댓값은 *V7* 의 120.59 까지 벌어진다. 타깃 *Class* 는 이진값(0/1)으로 분포는 2.6 에서 다룬다.

2.3 Feature 이름 특징

산탄데르 데이터셋은 *num_ind_saldo_imp_delta* 같은 접두사로 피처의 성격(건수·플래그·잔액·금액·변화량)을 유추할 수 있었다. 그러나 이 데이터셋의 *V1~V28*은 PCA로 익명화된 성분이라 이름 규칙이 존재하지 않는다. *V+* 번호는 주성분의 순번(분산 설명력 순서)일 뿐이며, 이름만으로는 어떤 거래 속성인지 전혀 알 수 없다. 따라서 접두사 분석·도메인 기반 피처 엔지니어링은 불가능하고, 이후 상관관계·feature importance 같은 통계적 지표로만 변수의 중요도를 해석했다. *Time Amount Class*만 원래 이름을 유지한다.

2.4 동일 값·중복 Feature

- **상수(constant) 컬럼:** 없음. 모든 피처가 분산 > 0 이므로 산탄데르처럼 “값이 하나뿐인 컬럼 제거” 단계는 필요 없었다.
- **중복 컬럼:** 없음. *V1~V28*은 PCA 성분이라 서로 직교(상관계수 절댓값이 모두 0에 수렴)해, 다른 컬럼과 값이 완전히 같은 경우가 없다.
- **행(row) 단위 완전 중복:** 존재한다. 31개 열 전체가 일치하는 완전 중복행이 1,081건(전체의 0.38%)이며, *Time*을 제외한 30개 열 기준으로는 9,144건까지 늘어난다. 이 행 중복은 데이터 누수와 직결되므로 3.5에서 별도로 다룬다.

2.5 ID 변수

이 데이터셋에는 산탄데르의 *ID*처럼 행을 식별하는 별도 컬럼이 없다. *Time*이 “첫 거래로부터 경과한 초”라 사실상 관측 순서를 담고 있으나, 실제 서비스 만족도나 사기 여부와 인과적으로 무관한 값이며 스케일도 다른 피처와 크게 달라(0~172,792), 팀 전원이 학습 피처에서 제외했다(근거는 4.4). 즉 학습에 쓰는 것은 *V1~V28*과 *Amount* 29개 피처뿐이다.

2.6 타겟 변수 분포 분석

*Class*는 0(정상) 284,315건 대 1(사기) 492건으로, 사기가 전체의 **0.173%**에 불과하다. 클래스 불균형이 매우 심하기 때문에 단순 정확도는 모든 거래를 정상으로 예측해도 99.8%에 도달하는 착시를 일으킨다. 따라서 팀 전체가 초기 EDA 단계부터 정확도 대신 ROC-AUC와 PR-AUC(또는 F1-Score)를 함께 확인하는 방향으로 지표를 통일했다(사기 비율 0.173% 데이터에서 ROC-AUC를 단독으로 쓰면 안 되는 이유는 7.2-9장에서 자세히 다룬다).

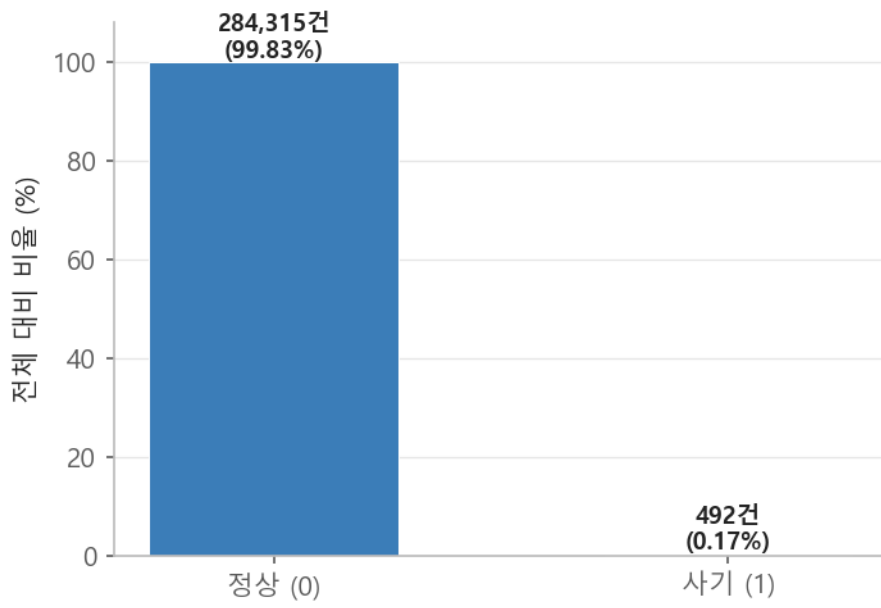


그림 1. Class 분포(정상 vs 사기). 사기 거래가 전체의 0.173%에 불과한 심각한 불균형을 보인다.

3. 탐색적 데이터 분석(EDA)

3.1 기술통계 분석

2.2 절의 *describe()* 결과에서 이미 확인했듯, $V1 \sim V28$ 은 평균이 0 에 수렴하고 표준편차만 $V1(1.96)$ 에서 $V28(0.33)$ 로 단조 감소한다. 이는 PCA 가 분산이 큰 순서로 성분을 매기기 때문이며, 뒤쪽 성분일수록 정보량이 적다. 다만 개별 성분의 꼬리는 매우 길어 최솟값 $V5 -113.74$, 최댓값 $V7 120.59$ 처럼 극단값이 관측된다.

*Amount*는 평균 88.35 에 비해 최댓값이 25,691, 표준편차가 250 으로 극단적인 오른쪽 꼬리를 보인다. 75% 값이 77.16 인데 최댓값이 그 300 배가 넘는다는 것은 소수의 고액 거래가 분포를 끌어당기고 있음을 뜻하며, 이는 3.6 절 로그 변환·스케일링 필요성의 근거가 된다.

3.2 상관관계 기반 주요 변수 분석

각 피처와 *Class*의 상관계수를 확인한 결과, 절댓값 기준으로 $V17 \cdot V14 \cdot V12 \cdot V10 \cdot V16 \cdot V3 \cdot V7 \cdot V11$ 이 사기 여부와 가장 높은 상관관계를 보였다(모두 0.15~0.33 수준). 이 중 $V14$ 는 사기($Class=1$) 케이스만 놓고 봤을 때 분포가 정규분포에 가장 가까워, 극단값을 이상치로 판정하기에 통계적으로 적합한 성질을 보였다. 이 성질을 활용한 이상치 탐지·제거 방법과 팀원별 적용은 3.3~3.4 절과 4.1 절에서 다룬다.

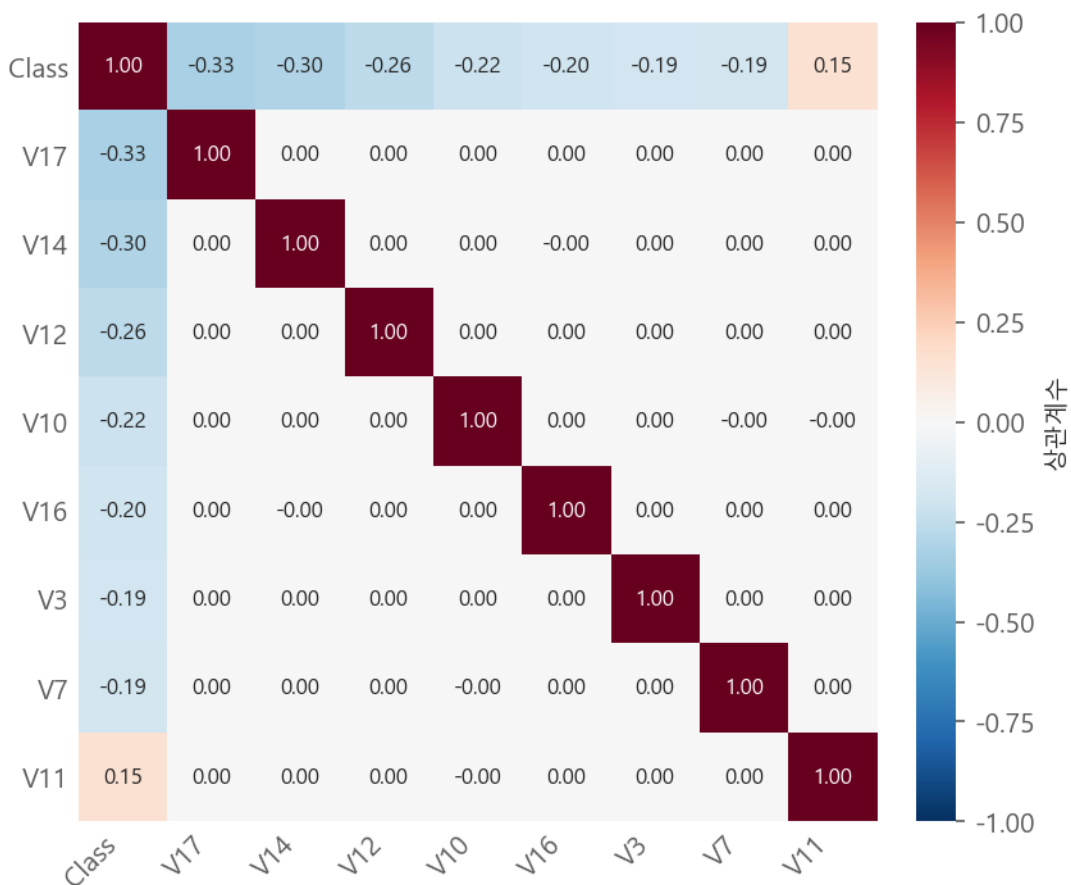


그림 2. *Class*와 상관관계가 높은 상위 8개 피처 히트맵(상관계수 절댓값 기준).

3.3 이상치 분석

이 데이터셋의 이상치는 산탄데르처럼 -999999 같은 “값으로 위장된 결측치”가 아니라, PCA 성분의 자연스러운 꼬리 끝에 있는 극단값이다. 팀은 사기(Class=1) 케이스만 대상으로, 상관관계가 높으면서 분포가 정규에 가장 가까운 **V14**를 이상치 판정 기준 피처로 삼았다.

사기 거래 492 건의 V14에 대해 IQR(사분위범위) 방식으로 하한을 구하면 다음과 같다.

- $Q1 = -9.69, Q3 = -4.28, IQR = 5.41$
- 하한선 = $Q1 - 1.5 \times IQR \approx -17.81$
- 하한선 아래로 벗어나는 사기 거래 **4건**이 이상치로 확인되었다.

정상 거래에는 이 기준을 적용하지 않았다. 정상 거래의 극단값까지 제거하면 오히려 학습에 필요한 정상 분포의 폭을 좁히게 되고, 목적이 “사기 케이스 중 학습을 방해하는 극단 4건”의 제거이기 때문이다.

3.4 변수별 이상치 시각화

사기(Class=1) 케이스만 대상으로 V14의 IQR을 구해 하한을 벗어나는 극단값을 제거하면, 아래 박스플롯처럼 사기 쪽 극단 이상치가 눈에 띄게 줄어든다.

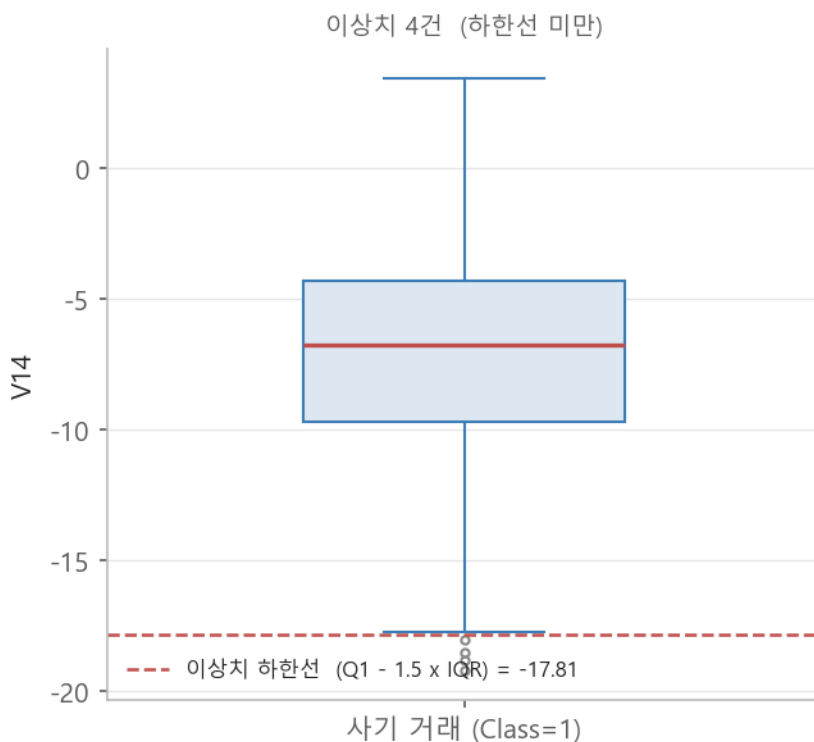


그림 3. 사기 거래(Class=1) 492 건의 V14 분포와 IQR 기반 이상치 하한선(약 -17.81). 하한선 아래 4건이 이상치로 제거된다.

3.5 중복 값 제거

컬럼 차원의 중복은 2.4에서 없음을 확인했고, 여기서는 행(row) 차원의 중복을 다룬다. `duplicated()`로 확인한 결과:

- 31 개 열이 전부 일치하는 완전 중복행: **1,081 건**(전체의 0.38%). 이 중 사기 거래가 32 건(전체 사기 492 건의 약 6.5%)으로, 정상 거래보다 중복 비율이 상대적으로 높다.
- *Time* 을 제외한 30 개 열(V1~V28·Amount·Class) 기준: **9,144 건**. *Time* 값이 초 단위로 대부분 서로 달라, 나머지 30 개 컬럼이 완전히 같은 행도 “중복 아님”으로 잘못 분류되기 때문이다. 28 개 PCA 성분이 소수점까지 일치하는 행은 우연이 아니라 같은 거래 레코드다.

완전히 동일한 값을 가진 행이 train/test 양쪽에 나뉘어 들어가면, 모델이 테스트셋 데이터를 학습 과정에서 이미 본 것과 같은 데이터 누수(data leakage)가 발생해 성능이 실제보다 부풀려진다. 실제로 이진희(XGBoost)는 중복을 제거하지 않았을 때 ROC-AUC 0.9916 이라는 높은 수치가 나온 것에 의문을 갖고 데이터 누수 가능성을 검토했고, train/test 분할 전에 전처리를 적용해 정보가 새고 있었음을 확인한 뒤 전처리 순서를 “분할 → 전처리”로 재구성했다.

Time 을 모델 입력에서 버리는 파이프라인이라면 중복 제거의 기준도 *Time* 을 뺀 30 개 열이어야 한다. 1,081 건만 제거하면 나머지 약 8,000 건은 모델이 보는 피처 벡터가 다른 행과 완전히 동일한데도 train/test 에 나뉘어 들어가 누수가 남는다. 본 프로젝트에서는 전유경·이진희가 31 개 열 전체 일치 기준(1,081 건)을, 김소현·권용우가 *Time* 제외 기준(9,144 건)을 적용했으며, 두 기준을 통일한 재실행은 향후 과제로 남긴다.

3.6 특정 컬럼 로그 변환

Amount 는 왜도가 큰 극단적 우측 편향 분포(3.1)를 가져, 값 범위가 다른 피처(대부분 -1~1 대)와 지나치게 차이 난다. 로지스틱 회귀·LightGBM 계열은 이 스케일 차이에 민감하므로, 평균·표준편차 기반의 StandardScaler 보다 중앙값·IQR 기반이라 이상치에 덜 흔들리는 **RobustScaler** 를 채택했다. 아래는 원본 분포와 RobustScaler 적용 후 분포를 로그 축에서 비교한 것이다.

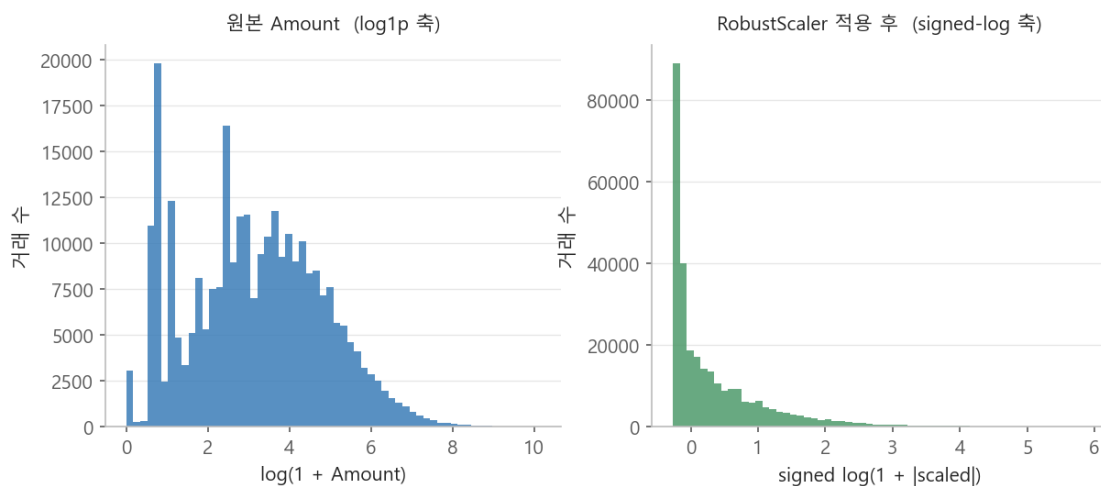


그림 4. 결제 금액(*Amount*) 원본 분포(좌)와 RobustScaler 적용 후 분포(우). 두 패널 모두 로그 축.

반면 CatBoost·XGBoost 같은 트리 기반 부스팅 모델은 임계값 분할 방식이라 단조 변환에 불변하므로, 전유경(CatBoost)은 스케일링을 아예 적용하지 않았다. 로그 변환·스케일링 적용 여부가 모델 계열에 따라 갈린 것이 이 프로젝트 전처리 전략의 특징이다(상세는 4.2·4.5).

4. 데이터 전처리

팀원 대부분이 “완전 중복행 제거 → V14 사기 케이스 이상치 제거 → Time 제거 → (선형·LightGBM 계열은) Amount RobustScaler”라는 공통 파이프라인을 기반으로, 각자의 모델 특성에 맞게 일부 단계를 가감했다.

4.1 이상치 처리

3.3 에서 정한 기준(사기 케이스 V14 IQR 하한 ≈ -17.81)에 따라, 하한 아래로 벗어나는 사기 거래 **4 건**을 제거했다. 다섯 명 전원이 이 단계를 공통으로 적용했다. 정상 거래에는 이상치 제거를 적용하지 않았다. 4 건은 전체 사기 492 건의 0.8%에 불과하지만, 학습 시 결정 경계를 극단으로 끌어당겨 나머지 사기 탐지를 방해하는 잡음이라 제거가 정당화된다.

4.2 분포 변환

*Amount*의 우측 편향을 완화하기 위해 RobustScaler 를 적용했다(그림 4). *V1~V28*은 이미 PCA 로 표준화된 성분이라 별도 변환을 하지 않았다. 로그 변환 자체는 RobustScaler 와 목적이 겹쳐 병행하지 않았다. 트리 기반 모델(CatBoost·XGBoost)은 임계값 비교만 사용하므로 이 변환을 의도적으로 생략했다 — 같은 데이터에 대해 모델 특성에 따라 전처리를 다르게 설계한 대표적 사례다.

4.3 파생 변수 생성

이 데이터셋은 *V1~V28*이 익명 PCA 성분이라 “고객별 거래 건수”, “최근 N 분 내 거래 횟수” 같은 도메인 기반 파생 변수를 만들 수 없다. 개별 거래 한 건을 29 차원 벡터로만 보는 구조라, 카드·고객 단위로 행을 묶는 집계 자체가 불가능하기 때문이다. 따라서 본 프로젝트에서는 파생 변수를 생성하지 않았다. 이 제약이 미탐지(FN)를 일정 수준 이하로 줄이지 못한 근본 원인이며(7.2), 향후 velocity 피쳐·그래프 기반 접근이 개선 방향으로 제안된다(9 장).

4.4 피쳐 정제 및 중요도 기반 가지치기

- **Time 제거**: 전원이 공통 적용. *Time*은 값 범위(0~172,792)가 다른 피쳐에 비해 지나치게 커서 로지스틱 회귀 계수 수렴에 문제(수렴 경고)를 일으킨다. 권용우의 실측 결과 *Time*을 제거하자 같은 조건에서 AUC가 0.9456 → 0.9663으로 개선되었다. 또한 3.5에서 다뤘듯 *Time*을 남기면 완전 중복행 탐지도 부정확해진다.
- **L1 정규화 기반 자동 변수 선택**: 김소현은 L1(Lasso) 정규화로 계수가 0이 되는 변수 약 10개(*V2·V3·V7·V9·V15·V17·V18·V24* 등)를 자동 제거했고, 오히려 성능이 개선(Test ROC-AUC 0.9719 → 0.9778)되는 것을 확인했다. 저신호 변수가 정밀도를 떨어뜨리고 있었던 것으로 해석된다(그림 9).
- **명시적 상수·중복 컬럼 제거**는 해당 없음(2.4).

4.5 스케일링 및 차원 축소

- **스케일링**: *Amount*에 RobustScaler(로지스틱 회귀·LightGBM). CatBoost·XGBoost는 미적용(4.2).
- **차원 축소**: 데이터가 이미 PCA로 축소·익명화된 상태이므로, PCA를 다시 적용하는 단계는 두지 않았다. *V1~V28*은 그 자체가 주성분이다.

4.6 데이터 분할

`train_test_split(test_size=0.2, stratify=y, random_state=...)`로 학습 80% / 테스트 20%로 나눴다. 사기 비율이 0.17%로 극단적이므로 `stratify`로 두 세트의 클래스 비율을 동일하게 유지하는 것이 필수다. 신뢰성 검증에는 5-Fold Stratified 교차검증을 함께 사용했다(이진희·김소현).

핵심 원칙은 **모든 전처리(스케일러 fit, 이상치 기준 계산, 중복 제거)를 분할 이후에 학습셋 기준으로만 수행한다**는 것이다. 분할 이전에 적용하면 테스트셋 정보가 학습에 새어 들어가 성능이 부풀려진다(3.5). 이진희·김소현·권용우가 각각 독립적으로 이 문제를 발견해 파이프라인을 “분할 → 전처리”로 바로잡았다.

5. 분석방법(Analysis method)

5.1 전처리 필요성 및 과정 개요

이 데이터셋에서 전처리가 프로젝트 전체의 쟁점이 된 이유는 두 가지다. 첫째, 피처가 익명 PCA 성분이라 도메인 지식을 넣을 여지가 없어, 성능을 끌어올릴 레버가 “데이터를 어떻게 손질하느냐”에 집중됐다. 둘째, 사기 비율이 0.17%로 극단적이라 중복행 한 줄, 이상치 몇 건 같은 작은 결정이 지표를 눈에 띄게 흔들었다.

팀 공통 파이프라인은 **완전 중복행 제거 → V14 사기 케이스 이상치 제거 → Time 제거 → (선형·LightGBM 계열은) Amount RobustScaler** 다. 여기에 도달하는 과정은 팀원마다 달랐는데, 다섯 명 모두 모델 튜닝에 앞서 전처리 결정을 A/B 로 비교했다.

담당 (모델)	전처리 비교·검증 방식
전유경 (CatBoost)	전처리 결정 3 건(중복 제거 / Time+Amount 제거 / Amount=0 제거)을 ROC-AUC-PR-AUC 로 A/B 비교. 이후 핵심 실험·이상 탐지·K-Fold 29 건을 같은 조건으로 전수 재검증. 스케일링은 트리 모델이라 배제.
권용우 (Logistic Regression)	원본 / 스케일링 / 이상치 제거 / 중복 제거를 $2 \times 2 \times 2 = 8$ 개 조합으로 전수 비교해 최적안 선정. 이후 “Time 을 먼저 제거해야 중복행이 정확히 잡힌다”를 발견하고 8 개 조합을 재실행.
김소현 (Logistic Regression)	class_weight vs SMOTE, L1 vs L2, ROC-AUC 기준 vs Average Precision 기준 C 재탐색 — 매 단계 두 가지 이상을 비교해 승자 채택.
이혜현 (LightGBM)	Baseline 대비 클래스 가중치·균형 샘플 앙상블·K-Fold 앙상블·Early Stopping·임계값 최적화·Hyperopt 등 7 가지 실험을 비교.
이진희 (XGBoost)	전처리 순서(분할 전/후)를 바꿔 데이터 누수를 직접 검증. 균형 샘플 앙상블·5-Fold 교차검증으로 신뢰성 재확인.

전처리 결정 하나하나가 지표에 미치는 영향은 6.4 의 그림 10(권용우 8 조합)에서 수치로 확인된다. 8 개 조합 모두 Test ROC-AUC 는 0.963~0.969 로 좁은 구간에 몰려 있으나 F1-Score 는 0.699~0.751 로 더 크게 갈렸고, ROC-AUC 가 가장 높은 조합과 F1 이 가장 높은 조합이 서로 달랐다 — 전처리 선택도 ROC-AUC 보다 F1·PR-AUC 로 판단해야 한다는 근거다.

5.2 사용 모델 설명

모델	담당	선택 이유
CatBoost	전유경	트리 기반 부스팅. 별도의 스케일링 없이 안정적으로 학습되고, oblivious tree-ordered boosting 으로 과적합 방지에 강점이 있다.
LightGBM	이혜현	리프 중심(leaf-wise) 성장으로 대용량 데이터에서 빠르고, <i>scale_pos_weight</i> 임계값 조정 등 불균형 대응 옵션이 풍부하다.
Logistic Regression	김소현 · 권용우	사기 확률을 함께 제시할 수 있고 계수로 해석이 가능하다. L1/L2 정규화로 과적합 억제·변수 선택, <i>class_weight</i> 로 불균형에 직접 대응할 수 있다. 두 명이 서로 다른 튜닝 경로로 접근해 결과를 교차 검증했다.
XGBoost	이진희	대표적 부스팅 모델. 사기:정상을 1:10 으로 나눈 여러 균형 샘플을 각각 학습해 예측 확률을 평균 내는 앙상블 방식으로 불균형에 대응했다.

5.3 하이퍼파라미터 최적화 기법

담당 (모델)	사용한 기법
CatBoost (전유경)	수동 그리드 — 클래스 가중치·판정 임계값· <i>eval_metric</i> 등 8 종 조합 비교
LightGBM (이혜현)	수동 실험 7 종 + Hyperopt(TPE, 50 회 탐색)
Logistic Regression (김소현)	GridSearchCV(C, roc_auc 기준) → Average Precision 기준 C 재탐색
Logistic Regression (권용우)	GridSearchCV vs Hyperopt(TPE, 15 회) 비교 + SMOTE + 임계값 최적화
XGBoost (이진희)	균형 샘플 앙상블(1:10) + Hyperopt(TPE, 100 회, 5-Fold CV 목적함수)

세 팀원(이혜현·이진희·권용우)이 수동/격자 탐색과 Hyperopt 를 나란히 비교했고, 그 결과는 7.3 에서 정리한다.

5.4 실행환경

- **언어·환경:** Python 3.10+, Jupyter Notebook (VS Code)
- **주요 라이브러리:** pandas, numpy, scikit-learn, xgboost(3.4), lightgbm, catboost(1.2), hyperopt(0.3), imbalanced-learn, matplotlib
- **재현성:** *train_test_split* 모델·교차검증에 *random_state*(대체로 0 또는 42)를 고정. 데이터는 팀 전원이 동일한 *creditcard.csv* 를 사용.

5.5 코드 재사용 및 함수화

팀원 5 명의 노트북을 전수 조사한 결과, 명시적으로 *def*로 정의된 공통 함수는 많지 않았다. 사기 케이스 IQR 이상치 인덱스를 구하는 *get_outlier(df, column, weight)*가 이해현·이진희·권용우 노트북에 거의 같은 형태로 등장했고, 혼동행렬과 주요 지표를 한 번에 출력하는 *get_clf_eval* 계열 리포팅 함수가 여러 노트북에 반복됐다. Hyperopt 를 쓴 세 명(이해현·이진희·권용우)의 *objective(params)* 함수도 세부 탐색 공간만 다를 뿐 "CV 점수 계산 → 음수 손실 반환 → *fmin*+TPE" 구조를 공유했다.

나머지 완전 중복행 제거·*Time* 컬럼 제거·*Amount* 스케일링·균형 샘플 앙상블 학습은 대부분 노트북 셀에 인라인으로 반복 작성되었다. 즉 "공통 함수"라기보다 "유사한 패턴을 각자 독립적으로 재구현"한 경우가 많았고, 이는 10 장 회고의 개선점("공통 전처리·평가 파이프라인을 프로젝트 초기에 표준화")과 직접 연결된다. 실제로 반복된 패턴을 정리해 재사용 가능한 형태로 다듬은 코드는 12 장 부록에 실었으며, 팀 노트북 5 개를 하나로 합친 통합본은 *02_classification_credit_card/팀_통합_creditcard.ipynb*에 있다.

6. 모델링

6.1 CatBoost (담당: 전유경)

CatBoost 는 트리 기반 부스팅 모델로 별도의 스케일링 없이도 안정적으로 학습되고 과적합 방지(oblivious tree, ordered boosting)에 강점이 있어 채택되었다. 전처리는 5.1 의 A/B 비교(중복 제거·Amount 관련 3 건)를 거쳐 “중복행 제거 → V14 이상치 제거 → Time 제거, 스케일링 없음”으로 확정했다. 기본 설정(*eval_metric='AUC'*)으로 학습한 기준선은 테스트셋 56,744 건(사기 93 건) 중 미탐지(FN) 13 건, 오탐(FP) 3 건을 기록했다. 이후 미탐지(FN)를 줄이기 위해 클래스 가중치·판정 임계값·평가지표 변경·트리 구조 튜닝 등 8 가지 방법을 실험했다.

실험	방법	FN	FP	Prec.	F1	ROC-AUC	상태
A (기준선)	<i>eval_metric='AUC'</i>	13	3	0.964	0.909	0.9759	기준
B	가중치=원비율(607.5)	9	977	0.079	0.146	0.9763	부작용 큼
C	가중치=SqrtBalanced	12	14	0.853	0.862	0.9832	절충
E (스윙 최선)	<i>scale_pos_weight</i> 스윙(5~100)	12	12	0.871	0.871	0.9717	한계
F (채택)	<i>eval_metric='F1'</i>	12	3	0.964	0.915	0.9743	채택
G	<i>depth=8, lr=0.05,</i> <i>iter=2000</i>	13	14	0.851	0.856	0.9748	효과 없음
H	<i>F1 + SqrtBalanced</i>	12	4	0.953	0.910	0.9785	시너지 없음

※ 실험 라벨 A~H 는 초기 시도 순서를 따른다. B 와 D 는 클래스 가중치를 원비율(607.5)로 적용한 동일 설정이라 결과가 같아 B 한 행으로 합쳤다(그래서 표에 D 행은 없다).

클래스 가중치를 손실 함수에 직접 반영하는 방식(B)은 FN 을 4 건 줄이는 대신 FP 를 977 건 만들어내 알림 피로(alert fatigue) 문제로 실무 투입이 불가능했다. *scale_pos_weight* 를 5~100 구간에서 스윙한 E 안은 그 방법 중에서는 가장 나았지만 FP 를 12 건까지밖에 못 줄여 F 안(FP 3 건)에는 미치지 못했다. 최종적으로 채택된 F 안은 모델의 학습 방식이나 데이터를 전혀 바꾸지 않고, CatBoost 가 학습 과정에서 저장하는 여러 스냅샷 중 “어느 iteration 을 최종 모델로 볼 것인가”의 기준만 AUC 에서 F1 로 바꾼 것으로, FP 를 3 건으로 그대로 유지하면서 FN 만 13 → 12 건으로 줄였다.

최종 채택 모델 성능: Confusion Matrix $\begin{bmatrix} 56648 & 3 \\ 12 & 81 \end{bmatrix}$ (그림 5), Precision 0.964 / Recall 0.871 / F1 0.915 / ROC-AUC 0.9743 / **PR-AUC 0.8898**(전유경이 재검증한 29 개 실험 중 최고 — 7.2 참조).

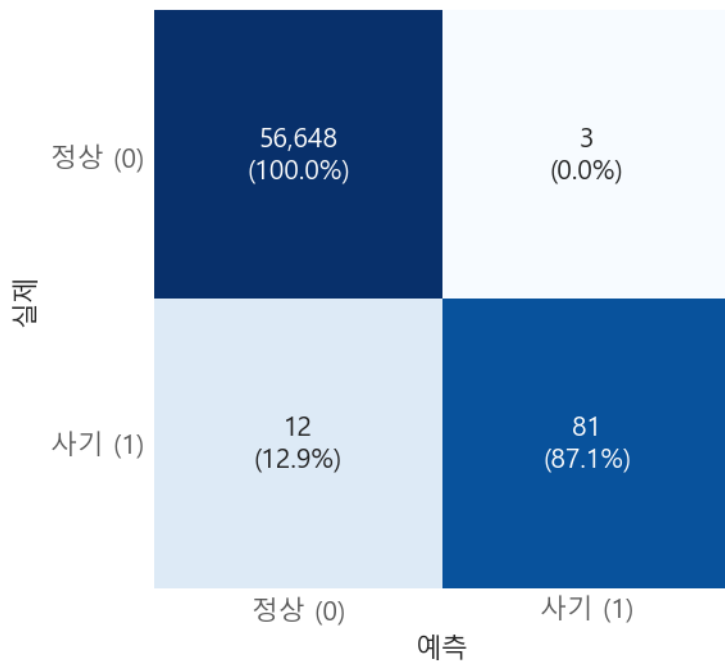


그림 5. CatBoost(F 안) Confusion Matrix. 색은 실제 클래스 내 비율, 괄호 안 수치도 같은 비율이다.

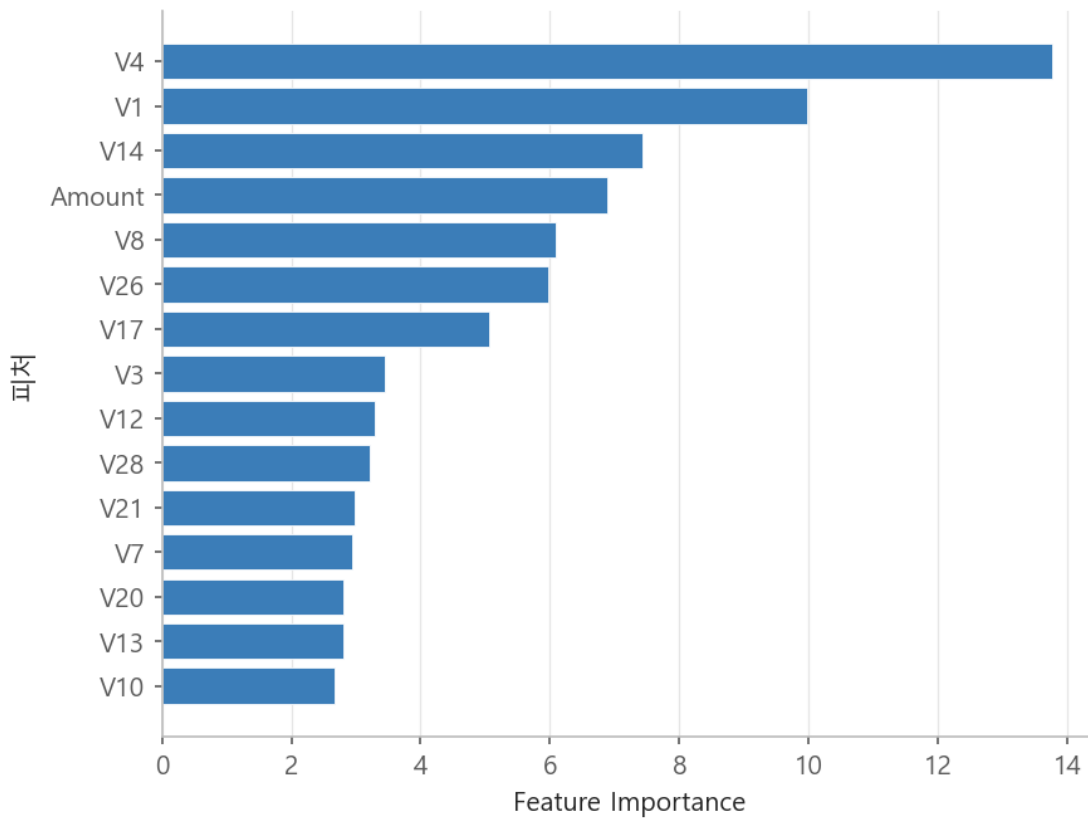


그림 6. CatBoost Feature Importance 상위 15 개.

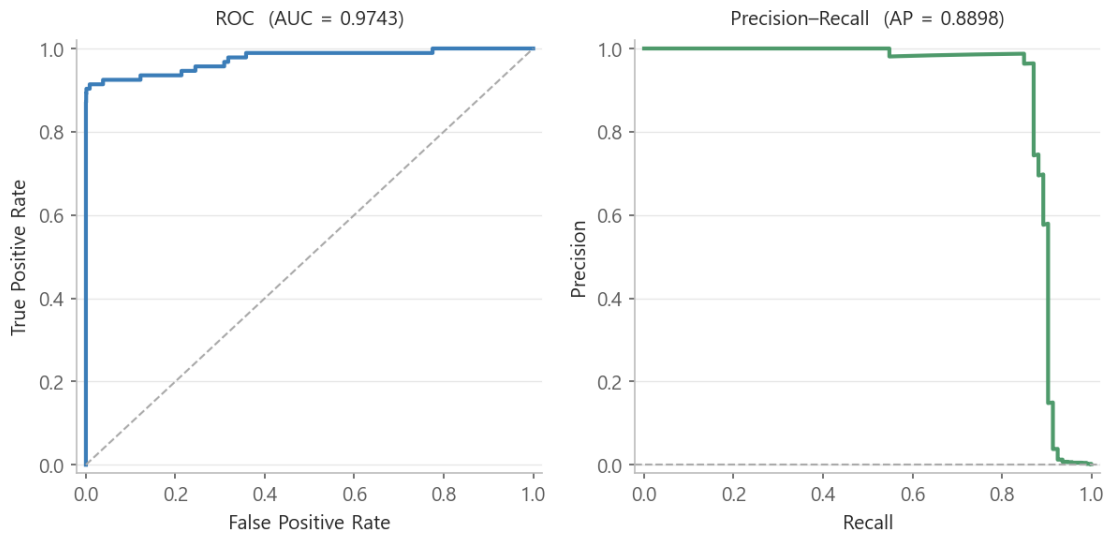


그림 7. CatBoost ROC Curve(좌)와 Precision-Recall Curve(우).

세 그림이 각각 짚어 주는 점은 다음과 같다. 혼동 행렬(그림 5)에서 오탐(FP)은 3 건으로 억제됐고 남은 오차는 사기 93 건 중 12 건을 놓친 미탐지(FN)에 몰려 있다. 피처 중요도(그림 6)는 V4·V1·V14·Amount 순으로 높게 나왔으나, V1~V28 이 익명 PCA 성분이라(2.2) “어떤 거래 속성이 사기와 연결되는가”까지는 알 수 없고 이상치 기준이던 V14 가 중요도 상위에도 든다는 통계적 확인에 그친다. ROC 곡선(그림 7 좌)은 좌상단에 바짝 붙어 AUC 0.9743 을 기록하지만, Precision-Recall 곡선(그림 7 우)은 recall 이 0.9 부근을 넘어서며 precision 이 가파르게 무너진다 — 극단적 불균형에서 두 곡선이 서로 다른 이야기를 한다는 점은 9 장에서 다룬다.

F 안 채택 이후에도 남은 미탐지(FN) 12 건이 튜닝만으로 더 줄지 않는 이유는 7.2 에서 별도로 분석한다.

6.2 LightGBM (담당: 이해현)

전처리(중복치·Time 제거 + V14 이상치 4 건 제거 + Amount RobustScaler)를 고정한 뒤, LightGBM 기준선 대비 7 가지 개선 실험을 비교했다.

실험	ROC-AUC	PR-AUC	F1-Score	비고
Baseline	0.9681	0.8414	0.8757	기준
클래스 가중치(scale_pos_weight)	0.9667	0.8345	0.8623	성능 소폭 하락
균형 샘플 앙상블(1:10, 60 개 모델)	0.9705	0.8412	0.6556	ROC-AUC 는 최고, F1 은 급락
K-Fold 앙상블(5-Fold)	0.8684	0.5663	0.6014	데이터 크기 대비 부적합
Early Stopping	0.7256	0.3396	0.5600	검증셋 분리로 학습량 부족, 큰 폭 하락
임계값 최적화(F1 기준)	0.9681	0.8414	0.8824	재학습 없이 F1 최고
Hyperopt 최적화(50 회 탐색)	0.9687	0.8421	0.8690	ROC-AUC-PR- AUC 동시 최고

균형 샘플 앙상블은 ROC-AUC 만 보면 가장 좋아 보이지만 F1 은 0.88 → 0.66 으로 크게 떨어져, 단일 지표만으로 판단하면 안 된다는 것을 보여주는 사례가 되었다. K-Fold 앙상블과 Early Stopping 은 이 데이터 규모(사기 375 건 내외의 학습 데이터)에서는 fold 당 사기 샘플이 지나치게 적어지거나 학습 데이터가 줄어드는 부작용이 더 커, 오히려 성능이 크게 후퇴했다. 최종적으로 전반적 순위 지표(ROC-AUC-PR-AUC)에서는 Hyperopt 튜닝안을, 실무에서 고정 임계값(0.5)을 그대로 쓸 경우에는 임계값 최적화(F1 0.8824)를 상황별 권장안으로 병기했다.

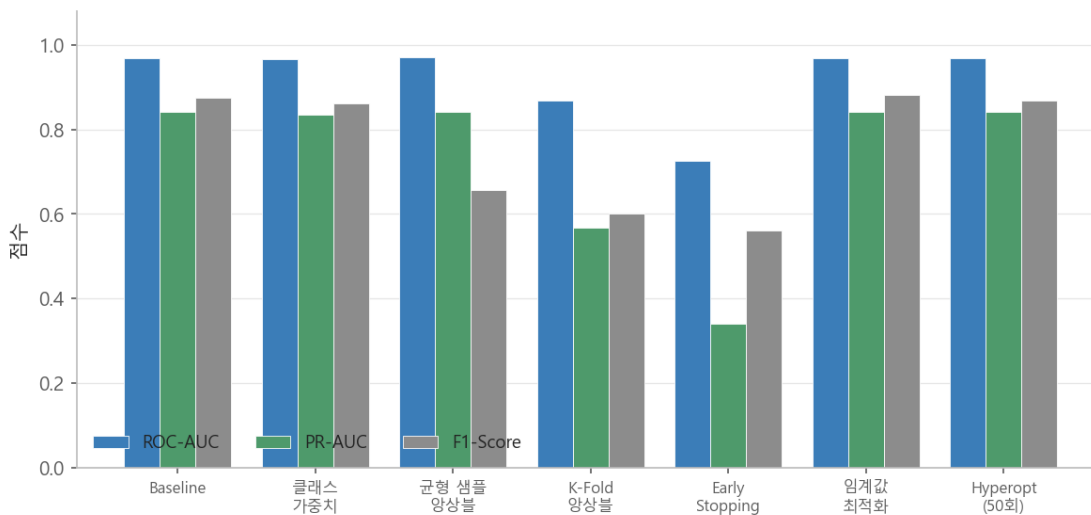


그림 8. 이해현의 LightGBM 7 가지 개선 실험별 ROC-AUC-PR-AUC-F1-Score. 균형 샘플 앙상블-K-Fold 앙상블-Early Stopping 은 F1-PR-AUC 가 크게 무너지며, 특히 균형 샘플 앙상블은 ROC-AUC 만 높게 유지돼 지표 선택의 함정을 보여준다.

6.3 Logistic Regression (담당: 김소현)

김소현은 `class_weight="balanced"`와 SMOTE 를 StratifiedKFold(5) 교차검증으로 비교해 `class_weight(CV ROC-AUC 0.9788)`가 SMOTE(0.9774)를 근소하게 앞서는 것을 확인하고 이를 최종 방식으로 채택했다. 이후 GridSearchCV 로 규제 강도(C)를 튜닝(Test ROC-AUC 0.9719)하고, L1(Lasso) 정규화로 계수가 0 이 되는 변수 약 10 개(V2·V3·V7·V9·V15·V17·V18·V24 등)를 자동 제거하면서 오히려 성능이 개선(Test ROC-AUC 0.9778)되는 것을 확인했다. 마지막으로 평가지표를 ROC-AUC 에서 PR-AUC(Average Precision)로 바꿔 C 를 재탐색하자 PR-AUC 가 0.6217 → 0.6606 으로 개선되었다(이 재탐색으로 Test ROC-AUC 는 0.9778 → 0.9701 로 소폭 내렸으나, 주 지표인 PR-AUC 를 우선한 선택이다). 추가로 이혜현의 LightGBM 실험을 자신의 파이프라인에서 재현하고 LR+LGBM 스택킹을 검증했으나, 스택킹(Test AP 0.8145)은 LGBM 단독(Test AP 0.8255)을 넘지 못했다 — 두 모델의 실력 격차가 클 때는 다양성보다 약한 모델이 발목을 잡는 효과가 더 크다는 것을 실증적으로 확인한 사례다.

김소현 최종 채택: L1 + Average Precision 기준 C 재탐색 / Test ROC-AUC 0.9701 / PR-AUC 0.6606.

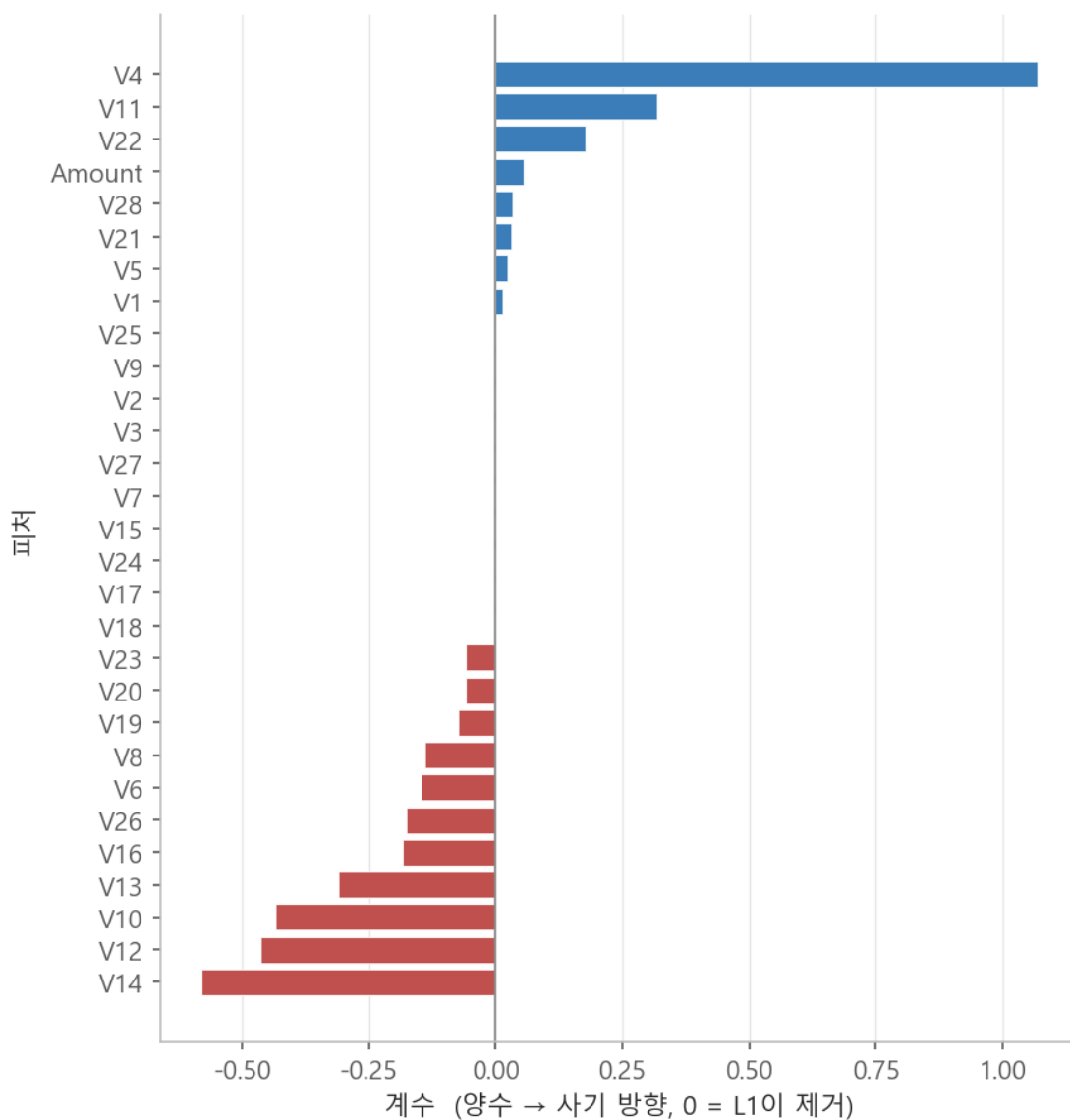


그림 9. 김소현의 Logistic Regression(L1 정규화) 피처별 계수. 계수가 0 에 붙은 변수는 L1 이 제거한 저신호 변수다(양수 → 사기 방향).

6.4 Logistic Regression (담당: 권용우)

권용우는 원본/스케일링/이상치 제거/중복 제거의 8 가지 조합을 전수 비교하는 것으로 시작해, Time 제거가 필요하다는 것을 발견한 뒤 전체 실험을 재실행했다. 8 개 조합 모두 Test ROC-AUC 는 0.963~0.969 로 좁은 구간에 몰려 있으나 F1-Score 는 0.699~0.751 로 더 크게 갈렸고, ROC-AUC 가 가장 높은 조합(원본+스케일링)과 F1 이 가장 높은 조합(이상치 제거+스케일링)이 서로 달랐다.

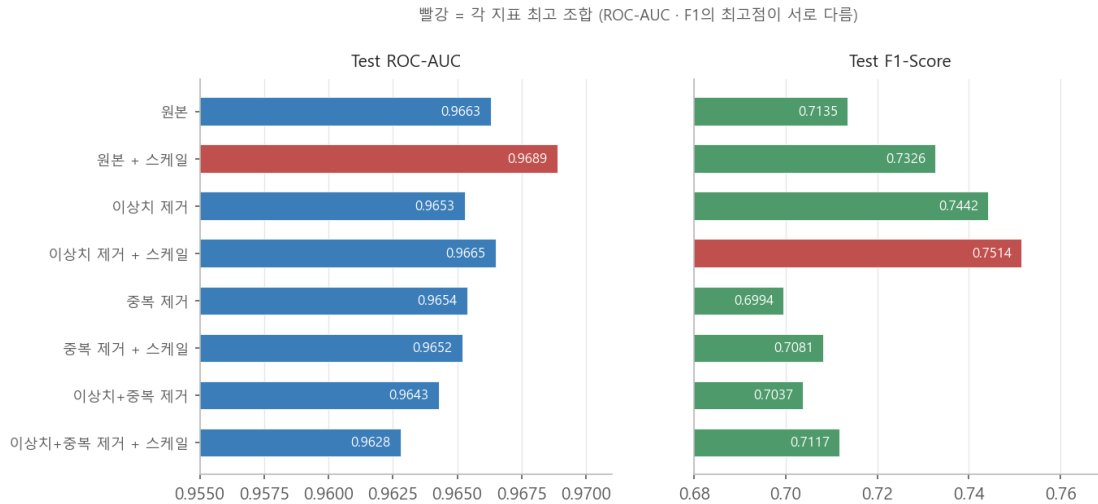


그림 10. 권용우의 Logistic Regression 8 가지 전처리 조합별 Test ROC-AUC(좌)와 F1-Score(우). 빨간 막대는 각 지표에서 가장 높은 조합. (Time 제거 후, 중복 9,144 건 기준)

이후 GridSearchCV(베스트 파라미터 $C=0.1$, $class_weight=None$, $penalty='l1'$, CV AUC 0.979)와 Hyperopt(TPE, 15 회 탐색, $C=0.878$, $penalty='l2'$, CV AUC 0.9771)를 동일한 데이터·탐색 범위에서 비교했다. 두 방법의 테스트 AUC 는 각각 0.9683, 0.9606 으로, L1 을 고른 GridSearchCV 가 L2 를 고른 Hyperopt 를 약 0.008 앞섰다 — 이 데이터에서는 탐색 방법(격자 vs 베이지안)보다 규제 종류(L1 vs L2) 선택이 결과를 갈랐고, Hyperopt 가 15 회 탐색 안에서 L1 영역을 충분히 훑지 못한 것으로 보인다. Hyperopt 최적 파라미터에 SMOTE 를 추가로 적용하고 F1 최적 임계값까지 탐색한 최종안은 F1 0.7771, AUC 0.9663 으로, 지금까지 시도한 모든 조합 중 F1 최고값을 기록했다.

권용우 최종 채택: Hyperopt($C=0.878$, l2) + SMOTE + 임계값 최적화 / Test ROC-AUC 0.9663 / F1 0.7771.

김소현·권용우 두 사람 모두 $class_weight$ /SMOTE 로 불균형을 보정하는 것이 ROC-AUC 보다 PR-AUC(또는 F1) 개선에 더 직접적으로 기여한다는 점, 그리고 로지스틱 회귀에서는 규제 강도(C)와 판정 임계값을 함께 조정해야 실질적인 성능 개선이 나타난다는 점에서 같은 결론에 도달했다.

6.5 XGBoost (담당: 이진희)

기본 전처리(V14 이상치 제거, RobustScaler, Time-Amount 원본 제거) 위에서, 사기:정상을 1:10 비율로 나누어 여러 균형 샘플을 만들어 각각 XGBoost 를 학습시킨 뒤 예측 확률을 평균 내는 앙상블 방식을 채택했다. 여러 모델을 빠르게 스크리닝하던 초기 단계에서 로지스틱 회귀만으로도 ROC-AUC 0.9898 이 나왔고, 트리 모델로 넘어가 XGBoost 에서 ROC-AUC 0.9916 까지 올랐으나, 지나치게 높은 수치에 의문을 갖고 데이터 누수 여부를 점검한 결과 train/test 분할 이전에 전처리(스케일러 fit, 이상치 기준 계산)를 적용한 것이 원인이었음을

확인했다. 이후 "분할 → 전처리" 순서로 파이프라인을 재구성하고 중복행까지 다시 제거한 뒤 XGBoost 를 재학습해 ROC-AUC 0.9773, PR-AUC 0.8150 을 얻었다.

신뢰성을 추가로 검증하기 위해 5-Fold Stratified 교차검증을 수행한 결과는 다음과 같다.

Fold	1	2	3	4	5	평균
ROC-AUC	0.9920	0.9838	0.9838	0.9764	0.9836	0.9839
PR-AUC(AP)	0.8408	0.8472	0.8264	0.8360	0.8499	0.8401

단일 분할 기준(ROC-AUC 0.9773, PR-AUC 0.8150)보다 5-Fold 평균(ROC-AUC 0.9839, PR-AUC 0.8401)이 두 지표 모두에서 오히려 안정적으로 높게 유지되어, 앞서 재구성한 파이프라인이 특정 분할에 우연히 잘 맞은 결과가 아니라는 것을 확인했다. 단일 분할 PR-AUC(0.8150)는 5-Fold 평균보다 약 0.025 낮아, 7.1 의 종합 비교에는 분할 편차가 작은 5-Fold 평균값을 사용했다. 이후 하이퍼파라미터 튜닝(Hyperopt, *max_depth* *min_child_weight* *colsample_bytree* *subsample* *learning_rate* 탐색, 100 회)을 시도했으나 오히려 CV ROC-AUC 가 소폭 하락(0.9728)해, 이 조합에서는 기본 파라미터의 앙상블 구조 자체가 이미 충분히 효과적이었다는 결론에 도달했다.

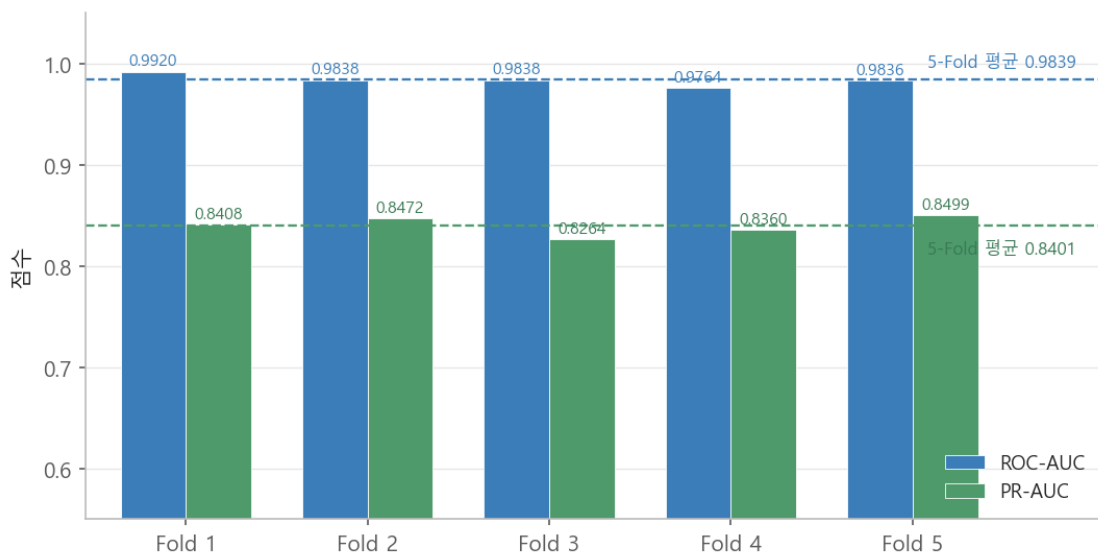


그림 11. 이진희의 XGBoost 5-Fold 교차검증 fold 별 ROC-AUC-PR-AUC. 다섯 fold 의 편차가 작고(ROC-AUC 표준편차 0.005, PR-AUC 0.008) 평균이 단일 분할값보다 높거나 같아, 재구성한 파이프라인이 특정 분할에 우연히 맞은 결과가 아님을 뒷받침한다.

7. 결과

7.1 모델별 최종 성능 비교

팀원마다 전처리와 검증 조건(단일 분할 vs 5-Fold, 채택 지표)이 서로 달라, 아래 표는 모델 간 순위를 매기지 않는다. 각자의 파이프라인 안에서 최종 채택된 모델의 성능으로만 읽어야 한다.

모델	담당	PR-AUC / AP	ROC-AUC	F1-Score	측정 조건
CatBoost	전유경	0.8898	0.9743	0.915	단일 분할
XGBoost	이진희	0.8401	0.9839	-	5-Fold 평균
LightGBM	이혜현	0.8421	0.9687	0.8690 / 0.8824	단일 분할
Logistic Regression	권용우	-	0.9663	0.7771	단일 분할
Logistic Regression	김소현	0.6606	0.9701	-	단일 분할

※ LightGBM F1 은 Hyperopt 튜닝안(0.8690) / 임계값 최적화안(0.8824)을 병기했다(6.2). “-”는 해당 팀원이 그 지표를 최종 보고에 포함하지 않은 경우다.

위 표의 수치는 서로 다른 조건(단일 분할 vs 5-Fold, 채택 지표)에서 측정된 값이므로 모델 간 절대 비교가 아니라 각자의 파이프라인 안에서 “무엇이 성능을 끌어올렸는가”를 읽는 자료다. 다만 PR-AUC 기준으로 트리 기반 부스팅 모델(CatBoost·XGBoost·LightGBM)이 로지스틱 회귀보다 앞서는 경향은 다섯 팀원의 실험 전반에서 공통으로 확인된다.

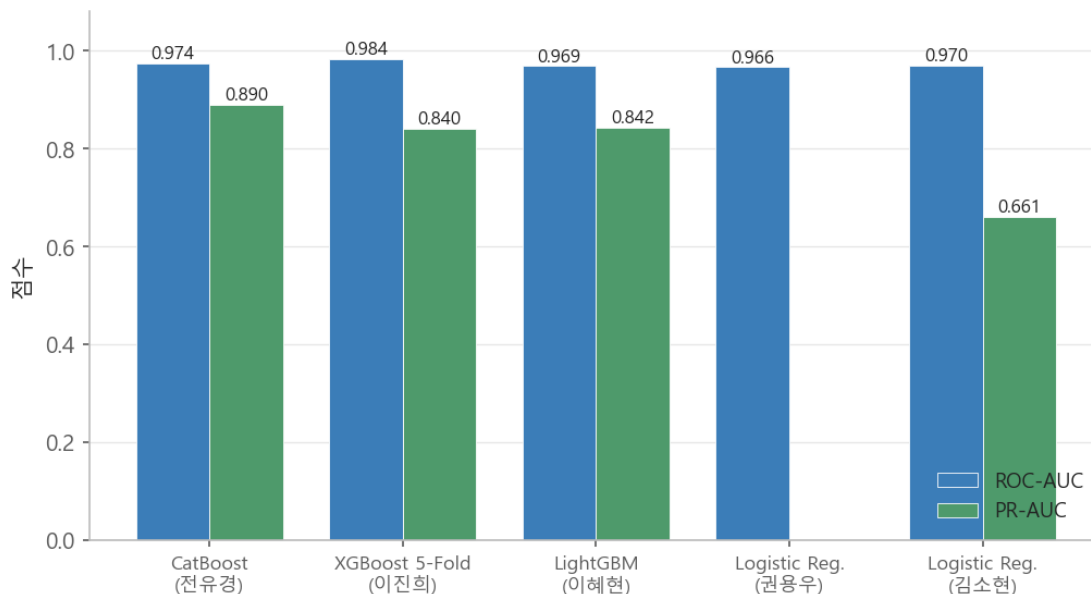


그림 12. 모델별 최종 채택 성능(ROC-AUC vs PR-AUC). PR-AUC 기준으로 트리 기반 부스팅 3 종이 로지스틱 회귀보다 높게 나타난다.

그림 12 에서 보듯 트리 기반 부스팅 모델이 로지스틱 회귀보다 PR-AUC 기준으로 뚜렷이 우수한 것은, V1~V28 이 이미 PCA 로 변환된 비선형적 조합 정보를 담고 있어 선형 결합에 의존하는 로지스틱 회귀보다

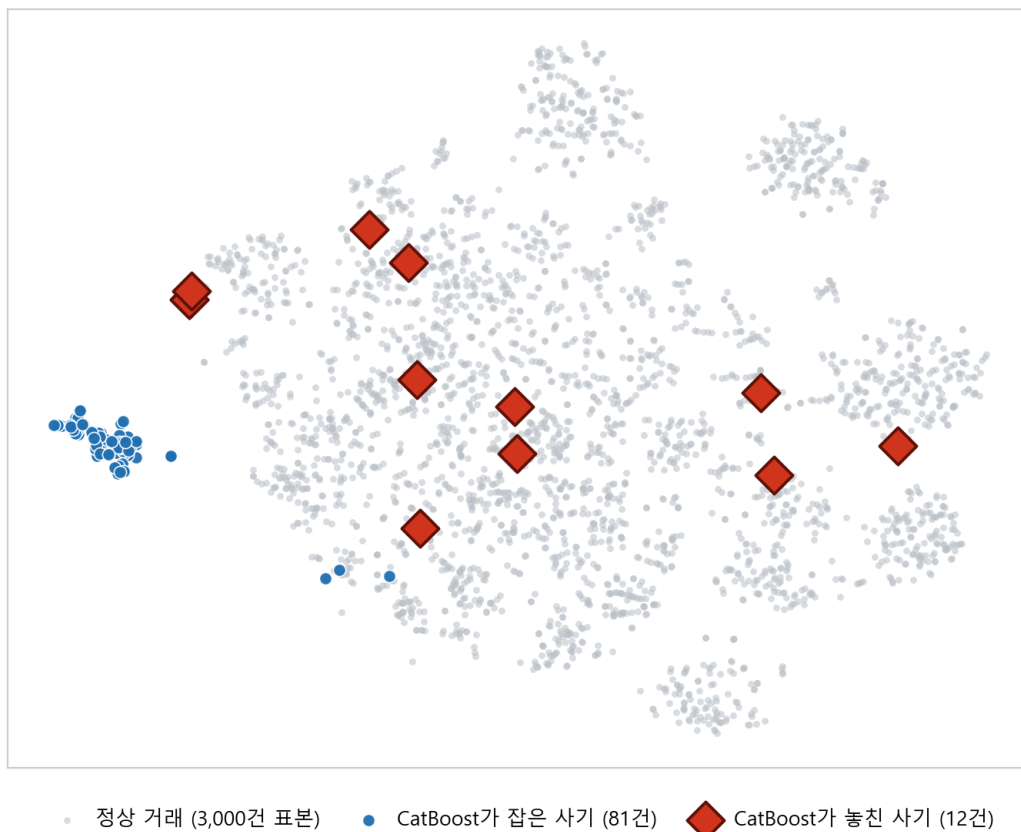
트리 기반 모델이 이 데이터의 비선형 패턴을 더 잘 포착하기 때문으로 해석된다. 다만 로지스틱 회귀는 “이 거래가 사기일 확률 92%”처럼 계수 기반으로 해석 가능한 예측을 제공하고 학습·추론 속도가 빨라, 트리 모델과는 다른 실무적 장점을 가진다.

7.2 최고 성능 모델 분석 — CatBoost 와 미탐지(FN)의 구조적 한계

PR-AUC 기준 최상위였던 CatBoost(F 안, PR-AUC 0.8898)에 대해, F 안을 채택한 뒤에도 남은 미탐지(FN) 12 건이 클래스 가중치·임계값·트리 구조를 아무리 조정해도 더 줄지 않은 이유를 규명했다.

먼저 최근접 이웃 거리를 측정한 결과, FN 12 건이 정상 거래와 떨어진 정도(중앙값 거리 2.49)는 CatBoost 가 실제로 잡아낸 사기 81 건(중앙값 거리 7.77)의 약 1/3 에 불과했다. 즉 놓친 12 건은 CatBoost 가 잡아낸 사기보다 정상 거래에 훨씬 가깝게 분포한다. 이어 Isolation Forest·LOF·One-Class SVM 등 라벨을 전혀 보지 않는 이상 탐지 기법 3 종을 CatBoost 예측과 결합했으나, 이 12 건 중 단 한 건도 추가로 잡아내지 못했다.

테스트셋 피쳐 공간 t-SNE 임베딩



파란 점(CatBoost가 잡은 사기)은 정상 거래 점군 가장자리·바깥쪽에 뚜렷한 무리를 이루는 반면, 빨간 다이아몬드(놓친 12건)는 대부분 정상 거래(회색) 점들 사이 한가운데 흩어져 있다 — 지도 위에서도 “이웃”이 온통 정상 거래뿐이다.

그림 13. CatBoost 테스트셋 피쳐 공간 t-SNE 임베딩. 파란 점(잡은 사기 81 건)은 정상 거래 점군 가장자리에 무리를 이루지만, 빨간 다이아몬드(놓친 12 건)는 정상 거래 점들 사이 한가운데 흩어져 있다.

학습 방식이 전혀 다른 지도학습(CatBoost)과 비지도 이상 탐지(3 종)가 정확히 같은 12 건에서 막혔다는 것은, 이 문제가 특정 모델의 튜닝 부족이 아니라 “거래 한 건을 29 차원 벡터로 보고 그 점의 위치만으로 판단한다”는 현재 피쳐 표현 자체의 정보량 상한에 부딪힌 결과로 해석된다.

카드·고객 단위의 맥락(velocity) 정보가 없는 완전 익명 데이터의 구조적 한계로, 모델을 더 튜닝하거나 다른 알고리즘으로 교체하는 것만으로는 해결되지 않는다.

한편 이 분석 과정에서 8 가지 핵심 실험에 전처리 A/B 테스트·이상 탐지 3 종·K-Fold 검증까지 더한 29 개 실험 전체를 PR-AUC 와 Confusion Matrix 기준으로 재검증했다. 이상 탐지 3 종은 ROC-AUC 가 0.95 대로 준수해 보이지만 PR-AUC 는 0.13~0.39 로 바닥에 붙어 있었고, F 안은 이 29 개 중 PR-AUC 0.8898 로 최상위였다. 다만 K-Fold 교차검증에서는 이 우위가 절대적이지 않아, F1 채택은 “결정 임계값 0.5 에서의 FN/FP”라는 실무적으로 중요한 지점을 최적화하는 선택이지 모든 임계값을 아우르는 전반적 랭킹 성능을 항상 최고로 만드는 선택은 아니다.

7.3 하이퍼파라미터 최적화 효과 분석

세 팀원(이혜현·이진희·권용우)이 각자의 모델에서 수동/격자 탐색과 Hyperopt(베이지안 최적화)를 나란히 비교했다. 지표는 모두 ROC-AUC 지만 검증 방식이 팀원마다 다르므로, 표는 각 행 안에서 수동 대 Hyperopt 만 비교한다(행 간 세로 비교는 의미가 없다).

담당 (모델)	검증 방식	수동 / GridSearchCV	Hyperopt	결과
이혜현 (LightGBM)	단일 분할 Test	기준선 ROC-AUC 0.9681	ROC-AUC 0.9687 (50 회)	Hyperopt 근소 우세, 탐색 공간 설계가 성능을 좌우
권용우 (Logistic Regression)	단일 분할 Test	GridSearchCV ROC-AUC 0.9683 (L1)	ROC-AUC 0.9606 (15 회, L2)	L1 을 고른 격자 탐색이 L2 를 고른 Hyperopt 를 약 0.008 앞섬
이진희 (XGBoost)	5-Fold CV	기본 파라미터 ROC-AUC 0.9839	ROC-AUC 0.9728 (100 회)	Hyperopt 오히려 하락, 기본 설정이 이미 충분

세 사례를 종합하면 서로 다른 세 검증 방식 어디에서도 Hyperopt 가 수동/격자 탐색을 압도적으로 앞서는 경우는 없었다. 데이터가 작고 불균형이 심할수록 탐색 공간을 단순하게 설계하는 것이 과도하게 넓은 공간을 베이지안 방식으로 탐색하는 것보다 효과적이었으며, 하이퍼파라미터 튜닝보다 판정 임계값 조정이나 평가지표(eval_metric) 선택 같은 “재학습이 필요 없는” 조정이 더 확실한 개선을 가져온 경우가 많았다.

7.4 피쳐 중요도 공통 인사이트

접근 방식이 다른 두 갈래(트리 중요도, 선형 계수)에서 반복적으로 상위에 오른 피쳐가 있다.

- **CatBoost feature importance**(그림 6): V4·V1·V14·Amount·V8 순.
- **Logistic Regression L1 계수**(그림 9, 김소현): 계수 절댓값 상위에 V14·V12·V10·V4 등이 위치하고, V2·V3·V7·V9·V15 등은 0 으로 눌러 제거됨.
- **Class 상관 상위**(그림 2): V17·V14·V12·V10·V16·V3·V7·V11.

세 리스트에 공통으로 등장하는 것은 **V14·V12·V10·V4** 다. 상관관계(선형 관계)·트리 중요도(비선형 분할 기여)·선형 계수(정규화 후 생존)가 서로 다른 개념인데도 같은 피쳐를 지목한다는 것은, 이 성분들이 사기

신호를 실제로 담고 있을 가능성이 높다는 뜻이다. 다만 V1~V28 이 익명이라 "그래서 어떤 거래 속성인가"까지는 이 프로젝트 범위에서 알 수 없다.

8. 종합 평가

데이터 이해도, 전처리의 타당성, 모델 비교 논리, 최적화 적용도, 결과 해석의 완성도 다섯 가지 관점에서 이 프로젝트를 자평하면 다음과 같다.

- **데이터 이해도 및 EDA의 깊이** — 익명 PCA 데이터라 접두사 분석·도메인 해석이 불가능한 조건에서, 상관관계·분포 정규성·행 단위 중복을 교차로 확인해 V14를 이상치 기준으로, 완전 중복행을 누수 요인으로 특정했다. 다만 개별 V 성분의 분포를 하나씩 시각화하는 작업까지는 하지 않았다.
- **전처리 방법의 타당성과 근거 제시** — 중복 제거·Time 제거·이상치 제거·스케일링 각각을 A/B로 비교해 ROC-AUC-PR-AUC 수치로 근거를 남겼다(5.1, 6.4). 모델 계열(트리 vs 선형)에 따라 스케일링 적용 여부를 다르게 가져간 판단도 근거가 명확하다. 한계는 중복 제거 기준(1,081 vs 9,144)을 팀 내에서 통일하지 못한 점이다.
- **모델 비교 논리와 지표 선정의 적절성** — 사기 비율 0.17%에서 ROC-AUC 단독 비교의 위험을 실측(이상 탐지 3종, 균형 샘플 앙상블)으로 보이고 PR-AUC·F1을 함께 채택했다. 팀원별 검증 조건이 달라 절대 순위를 매기지 않고 “측정 조건” 열과 함께 제시한 것도 과대 주장을 피한 선택이다.
- **최적화 적용 여부 및 성능 개선 정도** — 수동/격자/Hyperopt를 세 모델에서 나란히 비교했고(7.3), Hyperopt가 수동을 압도하지 못한다는 결론과 그 이유(탐색 공간 설계·데이터 규모)를 정리했다. 절대 성능 개선 폭은 크지 않았으나, “재학습 없는 조정(임계값·eval_metric)이 더 효과적”이라는 실용적 결론을 얻었다.
- **결과 해석의 명확성과 결론의 완성도** — 미탐지 12건이 모델 교체로 해결되지 않는 구조적 한계임을 지도·비지도 두 방법의 동일한 실패로 논증했고(7.2), 이를 바탕으로 다음 단계(velocity 피처·비용 민감 학습·확률 보정)를 개념 설명과 함께 제시했다(9장).

9. 결론

1. 극단적 불균형 데이터에서는 정확도·ROC-AUC 만으로 모델을 판단하면 안 된다. PR-AUC(또는 F1-Score, Precision/Recall)를 함께 봐야 실제 운영 성능을 가늠할 수 있다.
2. 중복행 제거와 train/test 분할 순서는 성능 수치 자체보다 우선하는 방법론적 원칙이다. 전처리를 분할 이전에 적용하면 실제보다 부풀려진 성능이 나오며, 이는 여러 팀원이 독립적으로 재현·확인했다. 특히 중복 제거는 Time 을 버리는 파이프라인이라면 Time 을 뺀 30 개 열 기준으로 해야 누수가 남지 않는다(세부는 3.5·10 장).
3. 익명 PCA 데이터에서도 타깃 상관과 분포 정규성을 함께 보면 신뢰할 만한 이상치 기준을 세울 수 있다. V14 는 Class 와 상관이 상위권이면서 사기 케이스 분포가 정규분포에 가장 가까워, 다섯 명 전원이 사기 케이스 기준 IQR 하한을 공통 이상치 기준으로 채택했다.
4. 트리 기반 부스팅 모델(CatBoost·LightGBM·XGBoost)이 로지스틱 회귀보다 PR-AUC 기준으로 우수했다. 다만 로지스틱 회귀는 해석 가능성과 속도 면에서 대체하기 어려운 장점을 유지한다.
5. 하이퍼파라미터 튜닝(Hyperopt 등)보다 판정 임계값 조정, 평가지표(eval_metric) 선택처럼 “재학습이 필요 없는” 조정이 더 확실한 개선을 가져오는 경우가 많았다. 클래스 가중치를 지나치게 강하게 적용하면 오탐이 폭증해 실무에 투입하기 어렵다는 점도 여러 모델에서 공통으로 확인되었다.
6. 이번 프로젝트에서 확보한 29 차원 익명 피쳐만으로는 미탐지(FN)를 일정 수준 이하로 줄일 수 없다는 것이 최종 결론이다. 학습 방식이 전혀 다른 지도학습(CatBoost)과 비지도 이상 탐지가 정확히 같은 사례에서 막혔고(7.2), 이는 모델·튜닝의 문제가 아니라 입력 정보량의 상한이다. 동시에, 트리 기반 부스팅이 로지스틱 회귀보다 PR-AUC 에서 앞서는 경향이 다섯 실험에서 일관되게 나타난 만큼, 익명 PCA 피쳐를 다루는 이런 문제에서는 부스팅 계열을 1 차 후보로 두는 것이 근거 있는 선택이다.

이 결론을 다음 단계로 잇기 위해, 팀 토의에서 아래 세 방향을 개선점으로 정리했다. 각 항목은 이번 프로젝트에서 다루지 않은 개념이므로 간단한 설명을 함께 붙인다.

- 맥락(velocity) 피쳐와 그래프 기반 이상 탐지. 거래를 독립된 한 점이 아니라 “카드·가맹점·기기가 시간축으로 연결된 흐름”으로 보고, 최근 N 분 내 거래 횟수·금액 급증, 동일 카드의 비정상 경로 같은 파생 변수를 만드는 것이 velocity 피쳐다. 더 나아가 거래를 노드-엣지로 이은 네트워크에서 여러 카드·계정이 얹힌 사기 링(fraud ring)을 찾는 그래프 기반 이상 탐지(graph anomaly detection)는 점 단위 판단의 한계를 구조적으로 넘는 접근으로, 팀이 아직 학습하지 않은 영역이다.
- 비용 민감 학습(cost-sensitive learning). 이번에는 class_weight 로 클래스 빈도만 보정했지만, 실무에서는 FN(놓친 사기 = 피해 금액)과 FP(막은 정상 거래 = 상담·이탈 비용)의 실제 비용이 다르다. 오분류마다 금전 비용을 부여하고 그 기대 비용이 최소가 되도록 손실 함수나 판정 임계값을 설계하는 방법을 익히면, F1·PR-AUC 같은 통계 지표를 넘어 비즈니스 목표에 바로 맞출 수 있다.
- 확률 보정(probability calibration). 권용우가 관찰했듯 SMOTE·class_weight 는 예측 확률을 위쪽으로 왜곡시켜 “사기 확률 90%”가 실제 신뢰도와 어긋나게 만든다. Platt scaling 이나

isotonic regression 으로 확률을 보정하면 임계값 결정과, 애매한 거래를 사람이 재검토하는
기준이 신뢰할 만해진다.

다섯 파이프라인을 동일 전처리·분할·교차검증·지표로 맞춰 공정 비교하는 작업은 위 개념들을
적용하기에 앞서 먼저 마무리한다.

10. 회고

10.1 잘된 점 · 아쉬운 점 · 향후 개선 방향

잘된 점

- 다섯 명 모두 ROC-AUC 대신 PR-AUC(또는 F1)를 주 지표로 삼아야 한다는 결론에 독립적으로 도달했고, 이를 교차 검증하는 과정에서 서로의 판단을 뒷받침하는 근거가 되었다.
- 데이터 누수(중복행 처리 순서, train/test 분할 전 전처리) 문제를 이진희·김소현·권용우가 각각 독립적으로 발견하고 파이프라인을 “분할 → 전처리” 순서로 바로잡았다 — 혼자 진행했다면 놓쳤을 문제를 서로의 실험 결과 비교를 통해 잡아낸 사례다.
- 같은 모델(Logistic Regression)을 두 팀원이 독립적으로 실험해 결과를 교차 검증할 수 있었고, CatBoost 사례에서는 지도학습과 비지도 이상 탐지를 결합하는 후속 분석까지 이어져 단순 성능 비교를 넘어선 원인 분석이 이루어졌다.

아쉬운 점

- CatBoost 의 미탐지(FN) 12 건은 여러 튜닝·이상 탐지 기법을 결합해도 끝내 줄어들지 않았고, 구조적 한계로 결론지었을 뿐 실제 해결책(카드·고객 단위 맥락 피쳐 추가 등)은 이번 프로젝트 범위에서 실행되지 않았다.
- 평가 파이프라인을 프로젝트 초기에 표준화하지 못했다. 최종 평가 지표(F1·PR-AUC·ROC-AUC 5-Fold 평균), 검증 방식(단일 분할 vs 교차검증), 중복행 제거 기준(완전 일치 1,081 건 vs Time 제외 9,144 건)이 팀원마다 달라, 7.1 비교표의 각 모델이 서로 다른 조건·서로 다른 크기의 학습 데이터 위에서 측정되었다. 그래서 7.1 은 순위를 매기지 않고 채택 결과만 나열하는 형태로 정리했으며, 다음 프로젝트에서는 이 세 가지를 초기에 공통 규칙으로 고정해야 한다.

향후 개선 방향

- CatBoost·LightGBM·XGBoost 세 모델을 스택킹 또는 가중 평균 앙상블로 결합하면 개별 모델의 PR-AUC 상한을 넘어설 가능성이 있다(김소현이 LR+LGBM 스택킹에서 확인했듯, 실력 격차가 큰 모델을 섞을 때는 가중치 설계가 관건이다).
- CatBoost 가 마지막까지 놓친 FN 12 건에 대해 라벨 노이즈 여부를 직접 검토하거나, 완전 자동 차단 대신 애매한 거래를 사람이 재검토하는 반자동 프로세스를 설계하는 것을 다음 단계로 제안한다.
- 팀 공통 평가 파이프라인(동일 지표·동일 검증 방식)을 프로젝트 초기에 표준화하면, 모델 간 비교의 공정성을 높이고 각자 유사한 평가 코드를 반복 작성하는 중복 작업도 줄일 수 있을 것이다.

10.2 팀원별 느낀점 및 피드백

아래는 팀이 작업 과정에서 실시간으로 기록해 온 *02_Creditcard.xlsx*의 팀원별 시트에 남아 있는 느낀점과 피드백을, 결과에 이르기까지 각자가 무엇을 느끼고 배웠는지의 기록으로 옮긴 것이다.

이혜현 — LightGBM. 불균형 사기 데이터에서는 무조건적인 하이퍼파라미터 튜닝보다 정밀한 전처리와 임계값 조정이 성능을 더 크게 좌우한다. 중복치를 제거하지 않았을 때 나온 높은 점수가 데이터 누수로 인한 착시였음을 확인하면서 올바른 전처리 순서의 중요성을 체감했다. 또한 LightGBM 의 하이퍼파라미터 탐색

공간을 지나치게 넓히면 교차검증 데이터에 미세 과적합되어 오히려 기존 튜닝안보다 점수가 낮아졌고, 소수 클래스 문제일수록 모델 복잡도를 적정 수준으로 제어해야 한다는 점을 배웠다. 결과적으로 복잡한 재학습 없이 예측 확률 임계값을 0.3~0.4 구간으로 낮추는 것만으로 오탐 증가 없이 재현율과 F1-Score 를 즉시 개선할 수 있었으며, 실무에서는 비즈니스 리스크를 반영한 사후 확률·임계값 조정이 핵심이라는 결론에 도달했다.

이진희 — XGBoost. 캐글 설명만으로 문제를 인지하던 이전 방식과 달리, 이번에는 데이터를 직접 열어 어떤 변수가 예측에 크게 작용하는지, 이미 PCA 로 정리된 데이터를 어떻게 표준화할지를 팀원과 논의하며 진행했다. 협업과 역할 분담이 이전 프로젝트보다 원활했고, 각자 아이디어를 내고 그 결과의 신뢰성을 어떻게 확보할지 함께 고민하는 과정이 특히 의미가 있었다. XGBoost 실험에서는 초기 점검만으로 ROC-AUC 0.99 대의 높은 수치가 나온 것에 의문을 갖고 데이터 누수 가능성을 반복 점검했으며, train/test 분할 이전에 스케일러 fit 과 이상치 기준 계산을 적용한 것이 원인임을 확인해 파이프라인 순서를 “분할 → 전처리”로 바로잡았다. 이 과정에서 높은 지표 자체보다 그 지표를 신뢰할 수 있게 만드는 검증 절차가 더 중요하다는 점을 배웠다.

김소현 — Logistic Regression. 로지스틱 회귀를 선택한 이유는 사기 여부를 0/1 로 분류하면서도 “이 거래가 사기일 확률”을 함께 제시할 수 있고, L1/L2 정규화로 과적합 억제와 변수 선택이 가능하며 class_weight 로 불균형에 직접 대응할 수 있기 때문이다. 스케일 민감성은 RobustScaler 로 완화했다. 실험을 진행하며 배운 점은 두 가지다. 첫째, class_weight 와 SMOTE 를 교차검증으로 비교했을 때 단순한 선형 모델에서는 class_weight 만으로 소수 클래스에 충분한 가중치를 줄 수 있어, SMOTE 가 오히려 소수 영역을 과도하게 확장한다는 것을 확인했다. 둘째, L1 정규화로 계수가 0 이 된 변수 10 개를 제거했더니 L2 대비 오히려 성능이 올라, 그 변수들이 노이즈에 가까워 정밀도를 떨어뜨리고 있었을 가능성을 확인했다. 평가지표를 ROC-AUC 에서 PR-AUC 로 바꿔 규제 강도(C)를 재탐색한 것도 같은 맥락으로, 극단적 불균형에서는 PR-AUC 가 실제 탐지 품질을 더 정확히 반영한다는 점을 실험으로 재확인했다.

전유경 — CatBoost. 데이터를 먼저 살펴보고 트리 기반 모델이 잘 맞을 것으로 예상했으나 결과는 달랐다. CatBoost 로 클래스 가중치·판정 임계값·평가지표·트리 구조까지 조정할 수 있는 방법을 모두 시도했지만 미탐지(FN) 건수가 일정 수준 아래로 내려가지 않았다. 처음에는 CatBoost 자체의 한계로 의심했으나, 라벨을 보지 않는 이상 탐지 기법 3 종을 결합해도 같은 사례에서 막히는 것을 확인하면서, 이는 특정 모델이 아니라 “거래 한 건을 29 차원 벡터의 한 점으로만 보고 판단하는” 지도학습 트리 방법론 전반의 구조적 한계라는 결론에 이르렀다. 카드·고객 단위의 맥락(velocity) 정보를 확보하는 것이 다음 과제라고 판단한다.

권용우 — Logistic Regression. 이전 과제와 달리 불균형이 극심한 데이터라, 전처리 조합이 성능에 미치는 영향을 정규분포 관점에서 체계적으로 검토했다. 이상치 제거·중복행 제거·스케일링을 독립 함수로 분리해 8 가지 조합을 동일 조건으로 비교했고, 그 과정에서 Time 컬럼을 남겨 두면 로지스틱 회귀 수렴이 방해되고 완전 중복 행 탐지도 부정확해진다는 점을 발견해 전체 실험을 Time 제거 버전으로 재실행했다. 파라미터 탐색은 시간 제약상 GridSearchCV 와 Hyperopt 를 비교하는 선에서 진행했으며, 두 방법의 결과 차이는 크지 않았다. SMOTE 나 class_weight 처럼 소수 클래스를 인위적으로 부풀리는 기법을 쓰면 예측 확률이 위로 쏠려 F1 최적 임계값이 1 에 가깝게 형성되는데, 이 경우 사기 확률이 90%로 나온 케이스도 정상으로 분류될 수 있어 F1 최댓값 지점이 실무적으로 안전한 기준은 아니라는 점을 확인했다. “재현율 하한을 정해 두고 그 안에서 정밀도를 최대화”하는 제약 기반 임계값 설정이 더 안전한 대안이라고 판단한다.

11. 참고문헌

1. Machine Learning Group – ULB. *Credit Card Fraud Detection*. Kaggle.
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
2. 권철민. 『파이썬 머신러닝 완벽 가이드』(개정판). 위키북스, 2022. — 불균형 데이터 처리, 평가지표(PR-AUC-F1), 부스팅 계열 모델 튜닝 절차 참고.
3. scikit-learn developers. *scikit-learn User Guide* — *train_test_split*, *StratifiedKFold*, *RobustScaler*, *LogisticRegression*, *precision_recall_curve*, *average_precision_score*. <https://scikit-learn.org/stable/>
4. CatBoost. *CatBoost Documentation* — *eval_metric*, overfitting detector. <https://catboost.ai/docs/>
5. LightGBM. *LightGBM Documentation* — *scale_pos_weight*, leaf-wise growth, early stopping.
<https://lightgbm.readthedocs.io/>
6. XGBoost. *XGBoost Documentation*. <https://xgboost.readthedocs.io/>
7. J. Bergstra, D. Yamins, D. D. Cox. "Hyperopt: A Python Library for Optimizing the Hyperparameters of Machine Learning Algorithms." *Proc. of the 12th Python in Science Conf.*, 2013.
8. imbalanced-learn developers. *imbalanced-learn Documentation* — SMOTE. <https://imbalanced-learn.org/>

12. 부록: 함수 코드

아래 함수들은 팀 노트북 5 개(이혜현·이진희·김소현·전유경·권용우)와 이를 합친 통합 노트북(02_classification_credit_card/팀_통합_creditcard.ipynb)을 조사한 결과, 실제로 *def*로 정의되어 여러 노트북에 공유된 것과, 셀에 인라인으로 반복 작성된 패턴을 재사용 가능하도록 정리한 것을 함께 담았다. 5.5 절에서 밝혔듯 팀 공통 함수화는 제한적이었으므로, 이 부록은 “이미 공유되어 있던 함수”와 “정리하면 공유할 수 있었던 패턴”을 함께 담았다. 각 함수의 docstring 에는 대응하는 본문 절 번호를 표시했다.

12.1 전처리 함수

완전 중복행 제거, 사기 케이스 V14 IQR 이상치 제거, *Time* 컬럼 제거, *Amount* RobustScaler 는 거의 모든 팀원의 노트북에 동일한 형태로 반복 등장한 패턴이다. 이 중 *get_outlier*는 이혜현·이진희·권용우 노트북에 실제로 *def*로 정의되어 있었고, 나머지는 셀에 인라인으로 반복된 것을 함수로 정리했다. 순서가 중요한데, *Time*을 먼저 버려야 나머지 30 개 열 기준으로 완전 중복행이 정확히 잡힌다(3.5 절).

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import RobustScaler

def get_outlier(df, column='V14', weight=1.5):
    """사기(Class=1) 케이스만 대상으로 IQR 기반 이상치 인덱스를 반환한다 (3.3 절).
    정상 거래에는 적용하지 않는다 - 정상 분포의 폭까지 좁히면 학습에 해가 된다."""
    fraud = df[df['Class'] == 1][column]
    q25, q75 = fraud.quantile(0.25), fraud.quantile(0.75)
    iqr = q75 - q25
    lower, upper = q25 - iqr * weight, q75 + iqr * weight
    return df[(df[column] < lower) | (df[column] > upper)].index

def preprocess_creditcard(df, scale=True):
    """팀 공통 전처리 1~4 단계(4.1·4.2·4.4 절)를 하나로 정리한 버전.
    분할 이후 학습 세트 기준으로만 호출해야 데이터 누수가 없다(4.6 절)."""
    X = df.copy()
    # 1) Time 제거: 스케일이 지나치게 커 로지스틱 회귀 수렴을 방해하고,
    #    남겨 두면 완전 중복행 탐지도 부정확해진다(3.5·4.4 절).
    X = X.drop(columns=['Time'], errors='ignore')
    # 2) 완전 중복행 제거 - Time 제거 후이므로 30 개 열 기준으로 잡힌다(3.5 절).
    X = X.drop_duplicates().reset_index(drop=True)
    # 3) 사기 케이스 V14 이상치 4 건 제거(3.3·4.1 절).
    X = X.drop(get_outlier(X, 'V14')).reset_index(drop=True)
    # 4) Amount RobustScaler - 트리 모델(CatBoost/XGBoost)에는 scale=False(4.2 절).
    if scale:
        X['Amount'] = RobustScaler().fit_transform(X[['Amount']])
    return X
```

12.2 공동 함수 — 모델링

*train_test_split(stratify=y)*로 계층 분할한 뒤(4.6 절), 이진희(XGBoost)·이혜현(LightGBM)이 공통으로 쓴 “사기:정상 = 1:N 균형 샘플을 여러 개 만들어 각각 학습 → 예측 확률 평균” 앙상블 흐름을 함수로 정리하면 다음과 같다.

```

from sklearn.model_selection import train_test_split

def split_stratified(df, target='Class', test_size=0.2, random_state=0):
    """계층 분할로 학습/테스트를 나눈다. 사기 비율 0.17%이므로 stratify 는 필수(4.6 절)."""
    X, y = df.drop(columns=[target]), df[target]
    return train_test_split(X, y, test_size=test_size,
                            stratify=y, random_state=random_state)

def make_balanced_samples(X_train, y_train, ratio=10, random_state=42):
    """사기: 정상 = 1:ratio 로 나눈 균형 샘플 데이터셋 리스트를 만든다(6.2·6.5 절)."""
    train = pd.concat([X_train, y_train], axis=1)
    fraud = train[train['Class'] == 1]
    normal = train[train['Class'] == 0].sample(frac=1, random_state=random_state)
    chunk = len(fraud) * ratio
    samples = []
    for start in range(0, len(normal), chunk):
        part = normal.iloc[start:start + chunk]
        if len(part) < chunk:
            break
        samples.append(pd.concat([fraud, part]).sample(frac=1, random_state=random_state))
    return samples

def ensemble_predict_proba(models, X_test):
    """여러 모델의 사기 확률을 평균 낸다 - 균형 샘플 앙상블의 최종 예측(6.5 절)."""
    return np.mean([m.predict_proba(X_test)[: , 1] for m in models], axis=0)

```

12.3 공동 함수 — 하이퍼파라미터 최적화

Hyperopt 를 쓴 세 명(이혜현·이진희·권용우)이 각자 독립적으로 작성한 *objective(params)* 함수는 세부 탐색 공간만 다를 뿐 "탐색 공간 정의 → 교차검증으로 평균 점수 계산 → 음수 손실 반환 → *fmin*+TPE 로 탐색"이라는 동일한 구조를 공유한다(5.3 절). 이를 모델에 무관하게 재사용 가능한 템플릿으로 일반화하면 다음과 같다.

```

from hyperopt import fmin, tpe, hp, Trials, STATUS_OK
from sklearn.model_selection import cross_val_score

def make_hyperopt_objective(model_cls, X, y, int_params=(), cv=5,
                            scoring='average_precision', fixed_params=None):
    """model_cls(XGBClassifier·LGBMClassifier·LogisticRegression 등)에 대해
    Hyperopt objective 를 생성한다. hp.quniform 은 float 를 반환하므로
    max_depth 등 정수 파라미터는 int_params 에 지정해 형변환한다.
    극단적 불균형이라 scoring 기본값은 PR-AUC(average_precision)로 둔다(2.6·9 장)."""
    fixed_params = fixed_params or {}
    def objective(params):
        p = {k: (int(v) if k in int_params else v) for k, v in params.items()}
        model = model_cls(**fixed_params, **p)
        score = cross_val_score(model, X, y, cv=cv, scoring=scoring, n_jobs=-1).mean()
        return {'loss': -score, 'status': STATUS_OK}
    return objective

def run_hyperopt(objective, space, max_evals=100, random_state=156):
    """fmin 실행을 감싸 베스트 파라미터와 베스트 CV 점수를 함께 반환한다."""
    trials = Trials()
    best = fmin(fn=objective, space=space, algo=tpe.suggest,
               max_evals=max_evals, trials=trials,
               rstate=np.random.default_rng(random_state))
    best_cv_score = -trials.best_trial['result']['loss']
    return best, best_cv_score

```

예를 들어 6.5 절의 XGBoost Hyperopt 실험(100 회 탐색)은 이 템플릿으로 다음과 같이 재현할 수 있다.

```

from xgboost import XGBClassifier

xgb_space = {
    'max_depth': hp.quniform('max_depth', 4, 15, 1),
    'min_child_weight': hp.quniform('min_child_weight', 1, 10, 1),
    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95),
    'subsample': hp.uniform('subsample', 0.5, 1.0),
    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2),
}
objective = make_hyperopt_objective(
    XGBClassifier, X_tr, y_tr,
    int_params=('max_depth', 'min_child_weight'),
    fixed_params={'n_estimators': 300, 'eval_metric': 'logloss', 'n_jobs': -1},
)
best_params, best_cv_ap = run_hyperopt(objective, xgb_space, max_evals=100)

```

12.4 공동 함수 — 평가

6 장 전체에서 “혼동행렬 + Precision/Recall/F1 + ROC-AUC + PR-AUC”를 한 번에 출력하는 패턴과, “튜닝 전 vs 튜닝 후” 또는 “모델 A vs 모델 B”의 점수를 나란히 비교하는 패턴이 반복되었다. 전자는 여러 팀원 노트북에 `get_clf_eval` 계열로 이미 존재했고, 후자는 7.1 절 종합 비교표에 대응하도록 리포팅 함수로 정리했다.

```

from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, average_precision_score, f1_score)

def get_clf_eval(y_test, pred, pred_proba):
    """혼동행렬 + 주요 지표를 한 번에 출력한다(5.5 절·6 장 전체에서 반복된 패턴).
    극단적 불균형이라 정확도는 출력하지 않고 PR-AUC(=AP)를 함께 본다(2.6 절)."""
    cm = confusion_matrix(y_test, pred) # [[TN, FP], [FN, TP]]
    roc = roc_auc_score(y_test, pred_proba)
    pr = average_precision_score(y_test, pred_proba) # PR-AUC (= AP)
    f1 = f1_score(y_test, pred)
    print("[TN, FP], [FN, TP] ="); print(cm)
    print(f"ROC-AUC {roc:.4f} | PR-AUC {pr:.4f} | F1 {f1:.4f}")
    print(classification_report(y_test, pred, digits=4))
    return {'roc_auc': roc, 'pr_auc': pr, 'f1': f1, 'cm': cm}

def best_threshold_f1(y_true, proba, grid=None):
    """F1을 최대화하는 판정 임계값을 찾는다 - 재학습이 필요 없는 조정(6.1 F 안·6.2·6.4 절)."""
    grid = grid if grid is not None else np.arange(0.05, 0.95, 0.01)
    scores = [(t, f1_score(y_true, (proba >= t).astype(int)))] for t in grid
    return max(scores, key=lambda x: x[1]) # (best_threshold, best_f1)

def report_scores(results: dict, primary='pr_auc'):
    """{'모델 이름': {'pr_auc': .., 'roc_auc': .., 'f1': ..}, ...} 형태를 표로 출력하고
    primary 지표 기준 최고 모델을 반환한다(7.1 절 종합 비교표에 대응)."""
    print(f"{'모델':24s}{ 'PR-AUC':>10s}{ 'ROC-AUC':>10s}{ 'F1':>10s}")
    for name, m in results.items():
        print(f"{name:24s}{m.get('pr_auc', float('nan')):>10.4f}"
              f"{m.get('roc_auc', float('nan')):>10.4f}{m.get('f1', float('nan')):>10.4f}")
    best = max(results, key=lambda k: results[k].get(primary, float('-inf')))
    return {'best_model': best, 'best_score': results[best].get(primary)}

```

`report_scores`는 예컨대 7.1 절 비교표를 `report_scores({'CatBoost': {'pr_auc': 0.8898, 'roc_auc': 0.9743, 'f1': 0.915}, 'XGBoost': {'pr_auc': 0.8401, 'roc_auc': 0.9839, ...}})`처럼 호출해 PR-AUC 기준 최고 모델을 바로 확인하는 용도로 쓸 수 있다.